OnBuch+

SIGNED COURSE • PREMIUM

ICT • UPPER SIXTH | UPPER SIXTH — ICT

CHAPTER 16

APPLY COMMON TECHNOLOGIES, FOR WEB DEVELOPMENT

SUGGESTED TIME: 2 PERIODS

IN THIS CHAPTER

This chapter introduces the two essential technologies that transform plain HTML pages into modern websites: CSS, which controls the visual presentation (colours, fonts, layout), and JavaScript, which adds interactivity and dynamic behaviour to web pages. Learners will style web pages and write scripts that react to user actions, skills directly examined at the GCE Advanced Level.

- By the end of this chapter I can explain the role of CSS and JavaScript in web development and how they complement HTML
- By the end of this chapter I can write CSS rules using selectors, properties and values
- By the end of this chapter I can apply CSS to a page using inline, internal and external stylesheets and explain the cascade and specificity
- By the end of this chapter I can style text, colours, backgrounds, borders and the box model of HTML elements
- By the end of this chapter I can insert JavaScript into a web page and use variables, data types and operators
- By the end of this chapter I can use control structures (if/else, loops) and functions in JavaScript

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

CONTENTS

1. Introduction to CSS: Syntax, Selectors and Integration
2. Styling with CSS: Text, Colours, Backgrounds and the Box Model
3. JavaScript Fundamentals: Syntax, Variables and Control Structures
4. Interactive Pages: DOM Manipulation, Events and Form Validation
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can explain the role of CSS and JavaScript in web development and how they complement HTML
- By the end of this chapter I can write CSS rules using selectors, properties and values
- By the end of this chapter I can apply CSS to a page using inline, internal and external stylesheets and explain the cascade and specificity
- By the end of this chapter I can style text, colours, backgrounds, borders and the box model of HTML elements
- By the end of this chapter I can insert JavaScript into a web page and use variables, data types and operators
- By the end of this chapter I can use control structures (if/else, loops) and functions in JavaScript
- By the end of this chapter I can manipulate the DOM to read and modify page content dynamically
- By the end of this chapter I can handle events (click, submit, keypress) to create interactive pages
- By the end of this chapter I can validate a simple HTML form with JavaScript before submission

PREREQUISITES

- Structure of an HTML document and common tags (html, head, body, headings, paragraphs, lists, links, images, tables, forms) from Lower Sixth
- Creating and saving files with extensions (.html, .css, .js) and opening them in a browser
- Basic notion of attributes and values in HTML tags
- Elementary algorithmic thinking: variables, conditions and loops (from the algorithmics lessons)
- Using a text editor such as Notepad++ or VS Code

1 INTRODUCTION TO CSS: SYNTAX, SELECTORS AND INTEGRATION

A young entrepreneur in Douala creates a website to sell her tailor-made dresses, but the pages look dull: plain black text on a white background, no colours, no layout, and nothing happens when a visitor clicks a button. Meanwhile, her competitor's site, built with CSS and JavaScript, displays attractive colours, well-organised sections and a form that instantly checks the customer's phone number before submission. Within weeks, the competitor's online orders triple while hers stagnate. What makes the difference between a static, unattractive page and a modern, interactive website? This chapter reveals the two technologies behind that difference: **CSS** for presentation and **JavaScript** for interactivity.

The problem is clear: HTML alone can only describe the content and structure of a page (headings, paragraphs, images, links). It was never designed to control appearance. In this first section, we discover how CSS solves this problem by separating content from presentation.

1.1 What is CSS? Definition and Role

Imagine building a house. The bricks, beams and doors give the house its structure — this is the role of HTML. But the paint, the tiles, the curtains and the decoration give the house its beauty — this is the role of CSS. A house without decoration is liveable but unattractive; a web page without CSS is readable but dull.

DEFINITION **CSS — Cascading Style Sheets**

CSS (Cascading Style Sheets) is a style language used to describe the presentation of a document written in HTML: colours, fonts, sizes, spacing, borders, backgrounds and layout. Its fundamental principle is the separation of content (HTML) from presentation (CSS): the HTML file says what is on the page, the CSS says how it looks.

DID YOU KNOW? **Why "Cascading"?**

The word cascading refers to the way styles "flow down" from several sources (browser defaults, external files, internal blocks, inline attributes) and combine according to priority rules, like water flowing down steps. We study this cascade at the end of the section. CSS is maintained by the W3C, the same consortium that standardises HTML.

1.2 Advantages of CSS

Why did the whole web adopt CSS instead of styling directly inside HTML tags? Consider a school website in Bamenda with 40 pages. If the colour of every heading is written inside the HTML of each page, changing the school's colour theme means editing 40 files. With CSS, one line in one file updates the whole site.

KEY POINTS **Main advantages of CSS**

- Consistency: all pages share the same look because they use the same stylesheet.
- Easier maintenance: one change in one file updates the entire website.
- Faster loading: the external CSS file is downloaded once and cached by the browser; HTML files stay small.
- Professional appearance: colours, fonts, spacing and layout give a modern, attractive design.
- Accessibility and flexibility: the same content can be presented differently for a phone, a projector or a printer.

1.3 Anatomy of a CSS Rule

CSS is made of **rules**. Each rule answers two questions: which element(s) do we style, and how?

DEFINITION **CSS rule**

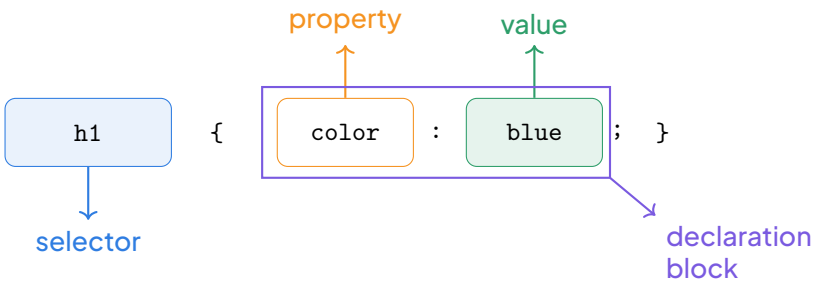
A **CSS rule** consists of a selector followed by a declaration block enclosed in braces `{ }`. Each declaration inside the block is a pair property: value; ended by a semicolon:

```
selector { property: value; property: value; }
```

EXEMPLE **A first CSS rule**

```
h1 { color: blue; font-size: 24px; }
```

This rule selects every `<h1>` heading and applies two declarations: the text colour becomes blue and the font size becomes 24 pixels.



Anatomy of a CSS rule: selector, declaration block, property and value.

WARNING **The forgotten semicolon**

A very common mistake is to forget the semicolon between two declarations:

```
h1 { color: blue font-size: 24px; }
```

The browser can no longer understand where the first declaration ends, so it ignores the broken declaration and all the following ones in the block. Always end each declaration with a semicolon — even the last one, as a good habit.

1.4 The Three Ways of Applying CSS

CSS can be attached to an HTML page in three ways. Each has its use, but they are not equally good.

1. Inline CSS (inside the tag, with the `style` attribute):

```
<p style="color: red;">Urgent notice</p>
```

It styles one single element. It is quick for a test but terrible for maintenance: imagine changing the colour of 200 paragraphs one by one.

2. Internal CSS (a `<style>` block in the `<head>`):

```
<head> <style> p { color: red; } </style> </head>
```

It styles one whole page. Useful for a single-page document, but the styles cannot be shared between pages.

3. External CSS (a separate `.css` file linked to every page): the styles live in a file such as `style.css`, and each HTML page points to it. This is the **recommended professional method**.

METHOD **Linking an external stylesheet**

To connect an external stylesheet `style.css` placed in the same folder as the HTML page:

1. Create a plain-text file named `style.css` and write your rules in it (no `<style>` tags inside!).
2. In the `<head>` of each HTML page, add exactly one line:
   `<link rel="stylesheet" href="style.css">`
3. Save both files and open the HTML page in a browser: the styles apply immediately.

The attribute `rel="stylesheet"` declares the relationship, and `href` gives the path to the file.

PROPERTY **Advantage of the external method**

An external stylesheet can be linked by any number of pages. Hence: one modification updates the whole site (maintenance), all pages look identical (consistency), and the browser downloads the CSS file only once (faster loading). (Stating this advantage is a classic GCE question — learn it.)

1.5 Basic Selectors

The selector is the "aiming device" of a rule. CSS offers several basic selectors:

Selector	Example	Selects
Element	`p`	every `<p>` paragraph
Class	`.promo`	every element with `class="promo"`
ID	`#menu`	the unique element with `id="menu"`
Universal	`*`	absolutely every element
Grouping	`h1, h2`	all `<h1>` **and** all `<h2>`

EXEMPLE **Selectors in action**

- `p { text-align: justify; }` — all paragraphs justified.
- `.price { color: green; }` — any tag carrying `class="price"` (a class can be reused on many elements).
- `#banner { background: yellow; }` — the single element with `id="banner"` (an id must be unique in the page).
- `h1, h2, h3 { font-family: Arial; }` — grouping: one rule for three element types.

WARNING **Class vs ID**

A frequent confusion at the GCE: a class (`.name`) may be applied to many elements, while an ID (`#name`) must identify one single element per page. Also note the syntax: classes use a dot, IDs use a hash, element selectors use no symbol at all.

1.6 The Cascade and Specificity

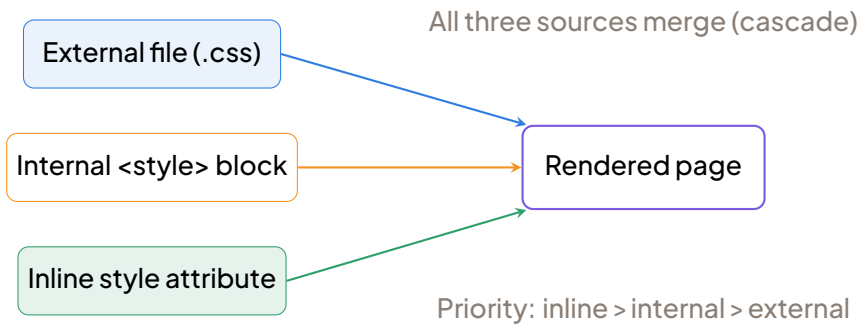
What happens when several rules target the same element with contradictory values? For example, an external file says `p { color: black; }` and an inline attribute says `style="color: red;"`. CSS resolves the conflict with two ideas: **specificity** (how precise the selector is) and **order** (the last rule read wins, all else being equal).

PROPERTY **Specificity hierarchy**

When several rules conflict, the winner is decided by this priority order, from strongest to weakest:

inline styles > IDs (#) > classes (.) > elements

For the three application methods, the priority is inline > internal > external. When two rules have equal specificity, the one that appears last in the code is applied. (This hierarchy must be known for the examination.)



The cascade: three sources of styles merge into one rendered page; on conflict, inline beats internal, which beats external.

WORKED EXAMPLE 1 **Which colour wins?**

A page contains: (1) an external file with `p { color: black; }`, (2) an internal block with `p { color: blue; }`, and (3) one paragraph written as `<p style="color: red;">Sale!</p>`.

State the colour of: (a) the “Sale!” paragraph; (b) the other paragraphs.

SOLUTION

Solution.

- (a) The “Sale!” paragraph carries an inline style. Inline has the highest priority (inline > internal > external), so it appears red.
- (b) The other paragraphs receive both the external rule (black) and the internal rule (blue). Internal beats external, so they appear blue.

1.7 CSS Comments and Good Practices

Like any code, CSS must be documented. A **comment** is a note ignored by the browser:

```
/* This rule styles the prices in green */
```

Comments start with `/*` and end with `*/` and may span several lines. Note carefully: HTML comments `<!-- ... -->` do **not** work inside a CSS file.

KEY POINTS **Good coding practices**

- Use an external stylesheet for any site with more than one page.
- Write one declaration per line and indent the block for readability.
- End every declaration with a semicolon.
- Choose meaningful class names: `.price`, `.menu`, not `.x1`.
- Comment the sections of your stylesheet (header, menu, footer…).

WARNING **Exam tip**

GCE Advanced Level questions very often ask you to name and distinguish the three ways of applying CSS (inline, internal, external) and to state one advantage of the external method. Answer with the exact vocabulary: “an external stylesheet can be shared by all pages, so a single change updates the whole site”.

EXERCISE 1 **Douala Dress Shop**

The Douala entrepreneur’s page contains `<h2 class="offer" id="main-offer">Weekend`

`Promotion</h2>`. The stylesheet contains: `h2 { color: black; }`, `.offer { color: blue; }` and `#main-offer { color: purple; }`.

1. What colour is displayed? Justify using the specificity hierarchy.
2. Write the line that links the external file `shop.css` to the page.
3. Give two advantages of CSS for her multi-page website.

SOLUTION **Exercise 1**

1. The heading appears purple: the ID selector (`#main-offer`) is more specific than the class (`.offer`), which is itself more specific than the element selector (`h2`).
2. `<link rel="stylesheet" href="shop.css">` placed in the `<head>`.
3. Any two of: consistency of design across all pages; easier maintenance (one file to edit); faster loading (the CSS file is cached); professional, attractive appearance.

2 STYLING WITH CSS: TEXT, COLOURS, BACKGROUNDS AND THE BOX MODEL

In the previous section we learnt the grammar of CSS: a rule is made of a **selector** and a **declaration block** containing property–value pairs, and we saw the three ways of integrating CSS into a page (inline, internal and external). Knowing the grammar is not enough: we must now learn the vocabulary, that is, the actual properties that make a page beautiful and readable. In this section we study the properties that control colours, text, backgrounds and, above all, the **box model** — the single most important idea in CSS layout, and a favourite of GCE Advanced Level examiners.

2.1 Colour in CSS

Observation. Open the website of a Cameroonian institution such as ANTIC (`antic.cm`) or a mobile money operator like MTN Mobile Money (`mtn.cm`) or Orange Money (`orange.cm`). You immediately recognise them by their colours: MTN's yellow (`#FFCC00`), Orange's orange (`#FF6600`). Every one of these colours is specified in CSS. There are three classic ways to write a colour, plus modern functional notations.

DEFINITION **Ways of writing a colour in CSS**

A colour may be written as:

1. a **colour name**: a keyword such as `red`, `blue`, `green`, `yellow`, `white`, `black` (about 140 standard names exist);
2. a **hexadecimal code**: `#RRGGBB`, where `RR`, `GG` and `BB` are two hexadecimal digits (00 to FF) giving the intensity of Red, Green and Blue respectively;
3. an **rgb() value**: `rgb(red, green, blue)` where each component is an integer from 0 to 255;
4. (modern) **rgba()** or **rgb() with alpha**: `rgba(red, green, blue, alpha)` or `rgb(r g b / alpha)` where `alpha` is a number between 0 (transparent) and 1 (opaque), allowing transparency.

EXEMPLE **Equivalent ways of writing the same colours**

Name	Hexadecimal	rgb()	Meaning
`red`	`#FF0000`	`rgb(255,0,0)`	maximum red, no green, no blue
`green`	`#008000`	`rgb(0,128,0)`	medium green (CSS named green)
`lime`	`#00FF00`	`rgb(0,255,0)`	maximum green
`white`	`#FFFFFF`	`rgb(255,255,255)`	all components at maximum
`black`	`#000000`	`rgb(0,0,0)`	all components at zero
`yellow`	`#FFFF00`	`rgb(255,255,0)`	red + green at maximum

Note that `#FF0000` and `rgb(255,0,0)` describe exactly the same colour, because FF in hexadecimal equals 255 in decimal. For transparency: `rgba(255,0,0,0.5)` gives a semi-transparent red.

Choosing readable contrasts. A colour choice is only good if the text remains readable. The rule of thumb is simple: **dark text on a light background, or light text on a dark background**. Yellow text on a white background, or dark blue text on a black background, strains the eyes and loses marks in a practical examination. A safe combination for body text is black or very dark grey (`#333333`) on white (`#FFFFFF` or `#FAFAFA`).

WARNING **Forgetting the # symbol**

A very common mistake is to write a hexadecimal colour without the hash symbol: `color: FF0000;` instead of `color: #FF0000;`. Without the `#`, the browser does not recognise the value and silently ignores the whole declaration — your text keeps its default colour and you may spend long minutes searching for the error. Always check: a hexadecimal colour begins with `#` and is followed by exactly six hexadecimal digits (0–9, A–F).

DID YOU KNOW? **Cameroon's digital colours**

MTN Cameroon's brand yellow is `#FFCC00` (rgb(255,204,0)), Orange Cameroon's orange is `#FF6600` (rgb(255,102,0)), and ANTIC uses a deep blue `#003399`. When you build a web page for a local client, using their exact brand colours (often found in their brand guidelines) shows professionalism. In the GCE practical, if a question asks for "MTN yellow", use `#FFCC00` or `rgb(255,204,0)`.

2.2 Text and Font Properties

Text is the heart of a web page, so CSS offers a rich family of properties to control it.

PROPERTY **Text and font properties most frequently examined**

Property	Role	Example values
`color`	colour of the text itself	`red`, `#003366`
`font-family`	the typeface (list of fallbacks)	`Arial, sans-serif`
`font-size`	size of the characters	`16px`, `1.2em`, `120%`
`font-weight`	thickness (bold or not)	`normal`, `bold`, `700`
`font-style`	italic or not	`normal`, `italic`
`text-align`	horizontal alignment of text	`left`, `center`, `right`, `justify`
`text-decoration`	underlining, line-through, overline	`underline`, `none`, `line-through`
`line-height`	vertical space between lines (unitless preferred)	`1.5`, `24px`
`text-transform`	capitalisation	`uppercase`, `lowercase`, `capitalize`
`letter-spacing`	space between characters	`1px`, `0.1em`

Units for font-size. Three units appear constantly:

- `px` (pixels): an absolute size, e.g. `font-size: 16px;` — precise and predictable;
- `em`: a size relative to the parent element's font-size. If the parent has `font-size: 16px`, then `1.5em` means 1.5 × 16 = 24 pixels;
- `%`: also relative to the parent's font-size: `120%` of 16px is 1.2 × 16 = 19.2 pixels.

Best practice: Set a base `font-size` on `body` (e.g. `16px`) and use `rem` (root em) or `em` for everything else, so the whole design scales if the user changes browser default size.

EXEMPLE **Styling a heading and a paragraph**

```
h1 { color: #003366; font-family: Arial, sans-serif;
\quad font-size: 2rem; text-align: center; font-weight: 700; }
p { font-size: 1rem; line-height: 1.6; text-align: justify; }
```

Every `<h1>` becomes dark blue, centred, in Arial at 2rem (typically 32px); every paragraph is justified with comfortable spacing between lines. Note the fallback list in `font-family`: if Arial is unavailable, the browser uses any sans-serif font.

WARNING **text-decoration: none on links**

By default, hyperlinks are underlined. To remove the underline, write `a { text-decoration: none; }`. A frequent exam slip is to try `underline: none` or `text-decoration: false` — neither exists. To add underline on hover: `a:hover { text-decoration: underline; }`.

2.3 Background Properties

The **background** of an element is the area behind its content and padding (but not the margin).

DEFINITION **Main background properties**

- `background-color`: paints the background with a colour, e.g. `background-color: #FFFFCC;`
- `background-image`: places an image behind the content, e.g. `background-image: url("banner.jpg");`
- `background-repeat`: controls tiling of the image: `repeat` (default, tiles in both directions), `repeat-x`, `repeat-y`, or `no-repeat` (image shown once).
- `background-position`: sets the starting position, e.g. `center center`, `top left`, `50% 50%`.
- `background-size`: `cover` (scales to cover entire area, may crop), `contain` (scales to fit entirely), or explicit sizes like `200px 100px`.
- `background-attachment`: `scroll` (default, scrolls with page) or `fixed` (parallax effect).
- Shorthand: `background: #FFF url("img.png") no-repeat center / cover;` sets colour, image, repeat, position, and size in one line.

EXEMPLE **A coloured banner with fallback**

`div.banner { background-color: #006600; background-image: url("logo.png"); background-repeat: no-repeat; background-position: center; color: white; padding: 20px; }` creates a green banner with a centred logo and white text. If the image fails to load, the green background ensures text remains readable. Always provide a `background-color` fallback when using images.

2.4 The CSS Box Model

Observation. Look at any web page: every element — a paragraph, an image, a heading, a table cell — occupies a rectangular region. CSS treats every element as a rectangle made of four nested layers. Understanding these layers explains why elements are spaced the way they are, and why two elements sometimes seem "too far apart".

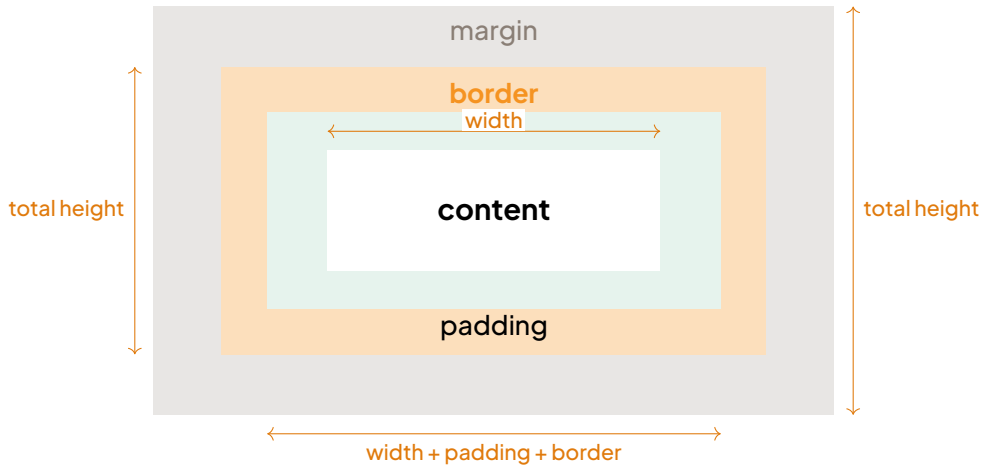
DEFINITION **The box model**

Every HTML element is a rectangular box composed of four layers, from the inside outwards:

1. the **content**: the text or image itself, whose size is set by `width` and `height`;
2. the **padding**: transparent space inside the box, between the content and the border;
3. the **border**: a visible (or invisible) line surrounding the padding;
4. the **margin**: transparent space outside the border, separating this box from its neighbours.

The total horizontal space occupied by an element is:

total width = margin-left+border-left+padding-left+width+padding-right+border-right+margin-right.



The CSS box model: content wrapped in padding, then a border, then the margin.

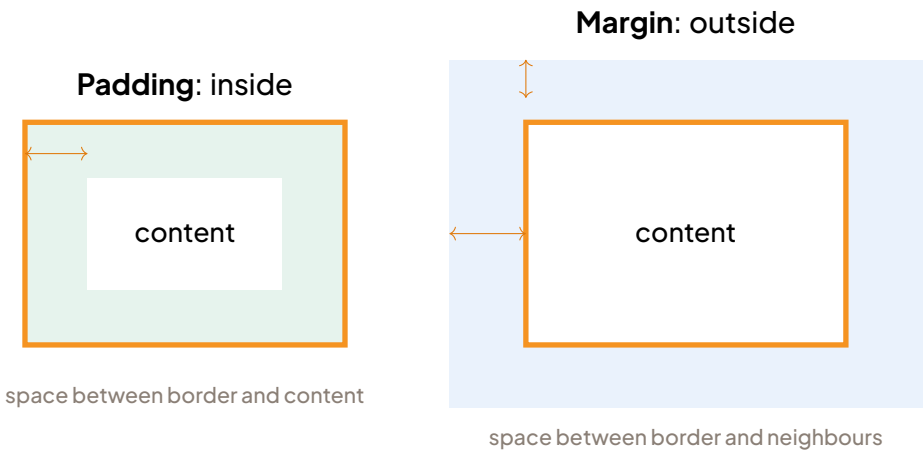
Borders. A border is described by three ingredients, which can be set separately or together with the shorthand `border`:

- `border-width`: thickness, e.g. `2px` or `thin`, `medium`, `thick`;
- `border-style`: `solid`, `dashed`, `dotted`, `double`, `groove`, `ridge`, `inset`, `outset`, `none`, `hidden`;
- `border-color`: any CSS colour;
- `border-radius`: rounds the corners, e.g. `border-radius: 10px;` — the larger the value, the rounder the corner. Can take up to four values (top-left, top-right, bottom-right, bottom-left).

Example: `border: 2px solid #003366;` sets width, style and colour in one line.

WARNING **Margin is outside, padding is inside**

The classic confusion: margin adds space outside the border (it pushes neighbours away), while padding adds space inside the border (it pushes the content away from the border). If you want more air between a paragraph's text and its frame, use padding; if you want more air between two framed paragraphs, use margin. Also remember: the background colour covers the content and the padding, but never the margin.



Padding (left) inflates the box from the inside; margin (right) separates the box from other elements.

2.5 The `box-sizing` Property (Critical for Layout)

By default (`box-sizing: content-box`), `width` and `height` apply only to the content area. Adding padding or border increases the total rendered size. This makes layout calculations painful.

PROPERTY **box-sizing: border-box**

With `box-sizing: border-box`, the declared `width` and `height` include content, padding, and border (but not margin). The content area shrinks to accommodate padding and border. This is the modern best practice.

METHOD **Applying border-box globally (recommended)**

Add this at the top of your stylesheet:

```
*, *::before, *::after {
\quad box-sizing: border-box;
}
```

Now `width: 300px; padding: 20px; border: 5px solid black;` results in a box that is exactly 300px wide (content becomes 250px). This prevents layout breaks when adding padding/borders.

2.6 Dimensions and Centring

Width and height. The properties `width` and `height` fix the size of the content area (or the whole

box if `box-sizing: border-box`). They accept pixels (`width: 600px;`), percentages of the parent (`width: 80%;`), or viewport units (`width: 50vw;`).

Min/Max constraints. `min-width`, `max-width`, `min-height`, `max-height` prevent elements from becoming too small or too large (responsive design).

Display. The `display` property controls how a box participates in the layout:

- `block`: the element occupies the whole available width and starts on a new line (e.g. `<p>`, `<h1>`, `<div>`, `<section>`); width, height, margin, padding all apply;
- `inline`: the element flows inside the text without line breaks (e.g. `<span>`, `<a>`, `<strong>`, `<img>`); width and height are ignored; vertical margins/paddings do not affect line flow;
- `inline-block`: flows inline like text, but accepts width, height, padding and margin like a block — very useful for menus and cards placed side by side;
- `none`: the element is removed from the layout entirely (not rendered).

METHOD **Centring a block horizontally**

To centre a block element (a `<div>`, a table, an image displayed as a block) inside its parent:

1. give the element a `width` smaller than its parent, e.g. `width: 80%;` or `width: 600px;`;
2. set the left and right margins to `auto`: `margin: 0 auto;` (top/bottom margin 0, left/right automatic).

The browser then shares the leftover space equally between the two sides, and the block sits exactly in the middle. Note: `text-align: center` centres text inside an element, not the element itself — do not confuse the two.

WARNING **Margin collapsing**

Vertical margins of adjacent block elements collapse: the larger margin wins, they do not add up. Example: `div{margin-bottom:30px;}` followed by `div{margin-top:20px;}` results in 30px gap, not 50px. Horizontal margins never collapse. This is a frequent exam trap.

2.7 Styling Tables and Forms

Unstyled HTML tables have no visible borders and cramped cells; unstyled forms look messy. A few properties transform them.

- `border-collapse: collapse;` on the `table` merges the double borders of cells into single clean lines;
- `border` and `padding` on `th` and `td` give each cell a frame and breathing space;
- `background-color` on `th` distinguishes the header row;

- **(exam-useful extra)** the pseudo-class `:nth-child(even)` selects every even row: `tr:nth-child(even) { background-color: #E8F0FE; }` produces alternating row colours ("zebra striping"), which greatly improves readability of long tables such as class lists or marks;
- `:hover` on `tr` highlights the row under the mouse: `tr:hover { background-color: #D0E0FF; }`;
- for forms, give `input`, `select`, `textarea` a `padding`, a `border`, a `border-radius`, and a consistent `font-size` (prevents zoom on iOS); align labels with `display: block` or flexbox.

WORKED EXAMPLE 2 **Styling a marks table**

Write the CSS so that a table of student marks has collapsed single grey borders, a dark blue header row with white text, cells padded by 8 pixels, centred text, and a light background on every even row. The table should be centred on the page at 80% width.

SOLUTION

```
table {
\quad border-collapse: collapse;
\quad width: 80%;
\quad margin: 20px auto;
}
th, td {
\quad border: 1px solid #999;
\quad padding: 8px;
\quad text-align: center;
}
th {
\quad background-color: #003366;
\quad color: white;
}
tr:nth-child(even) { background-color: #E8F0FE; }
tr:hover { background-color: #D0E0FF; }
```

Explanation. `border-collapse: collapse` avoids doubled lines between cells; the selector `th, td` applies the same border and padding to both header and data cells; the table is centred with `width` plus `margin: 0 auto`; `:nth-child(even)` paints rows 2, 4, 6, … giving the zebra effect; `:hover` adds interactivity.

2.8 Mini-Workshop: A School Notice-Board Page

EXPERIMENT **Styling the GBHS notice board**

A school notice-board page contains: a main heading, a short introduction paragraph, a table of announcements, and a highlighted "urgent" section. Here is a complete internal stylesheet demonstrating all concepts from this section:

```
* { box-sizing: border-box; }
body { font-family: Arial, sans-serif; background-color: #F5F5F5; margin: 0; padding: 20px;
h1 { color: #003366; text-align: center; margin-bottom: 0.5em; }
p.intro { font-size: 1.1rem; line-height: 1.6; text-align: justify; color: #333;
table { border-collapse: collapse; width: 90%; margin: 20px auto; }
th, td { border: 1px solid #666; padding: 10px; text-align: left; }
th { background-color: #003366; color: white; }
tr:nth-child(even) { background-color: #E8F0FE; }
tr:hover { background-color: #D0E0FF; }
div.urgent { background-color: #FFF0F0; border: 2px solid #CC0000;
\quad border-radius: 8px; padding: 20px; margin: 20px auto; width: 80%; }
div.urgent h2 { color: #CC0000; margin-top: 0; }
```

Observe how every idea of this section appears: named and hexadecimal colours, relative font sizes (`rem`), the box model (padding inside the urgent box, margin outside it), rounded corners, centring with `margin: auto`, `box-sizing: border-box` for predictable sizing, and a zebra-striped table with hover effect. Try it, then change one property at a time and observe the effect in the browser — this is the fastest way to master CSS.

KEY POINTS **Key points**

- Colours: names, `#RRGGBB` (never forget the #), `rgb(r,g,b)`, `rgba(r,g,b,a)`; always keep a strong text/background contrast (WCAG AA minimum 4.5:1).
- Text: `color`, `font-family` (with fallbacks), `font-size` (px, em, rem, %), `font-weight`, `font-style`, `text-align`, `text-decoration`, `line-height` (unitless), `text-transform`.
- Backgrounds: `background-color`, `background-image: url(...)`, `background-repeat`, `background-position`, `background-size`, shorthand `background`.
- Box model: content → padding → border → margin; total width adds all horizontal layers.

- `box-sizing: border-box` makes `width` include padding and border — use it globally.
- Padding is inside the border, margin is outside; background covers content+padding only.
- Centre a block with a `width` plus `margin: 0 auto`; `text-align: center` centres inline content.
- `display`: block, inline, inline-block, none; margin collapsing vertically.
- Tables: `border-collapse: collapse`, zebra stripes with `tr:nth-child(even)`, hover highlighting.

EXERCISE 2 **Practice**

A `<div>` has `width: 200px; padding: 10px; border: 5px solid black; margin: 20px;`. Assume `box-sizing: content-box` (default). (a) Calculate the total horizontal space it occupies. (b) Write the CSS to centre it horizontally while keeping the 20px top/bottom margins. (c) Write a rule making every odd row of a table light yellow (`#FFFFCC`). (d) If we add `box-sizing: border-box` to the div but keep `width: 200px`, what becomes the content width?

SOLUTION **Exercise 1**

(a) Total width = 20 + 5 + 10 + 200 + 10 + 5 + 20 = 270 pixels. (Left margin + left border + left padding + width + right padding + right border + right margin). (b) `div { width: 200px; margin: 20px auto; }` (top/bottom 20px, left/right auto). (c) `tr:nth-child(odd) { background-color: #FFFFCC; }`. (d) With `border-box`, the declared width (200px) includes padding and border. Content width = 200 − 2 × (10 + 5) = 170 pixels.

3 JAVASCRIPT FUNDAMENTALS: SYNTAX, VARIABLES AND CONTROL STRUCTURES

In the previous section we learnt how CSS dresses a web page: colours, fonts, backgrounds and the box model control the presentation of the content. But a page made only of HTML and CSS is static: it always displays the same thing, whatever the visitor does. If you want a page that reacts — a button that computes the average of your mock GCE marks, a form that refuses an empty telephone number, a counter that increases when you click — you need a third technology: **JavaScript**. HTML gives the structure, CSS gives the style, and JavaScript gives the behaviour.

3.1 What is JavaScript?

DEFINITION **JavaScript and script**

JavaScript is a scripting language executed by the web browser (Chrome, Firefox, Edge…) on the visitor's computer. It allows a web page to become dynamic and interactive: it can read what the user types, perform calculations, modify the content of the page and react to events such as clicks. A **script** is a short program, written as a sequence of instructions (statements), that the browser executes line by line.

DID YOU KNOW? **JavaScript is not Java**

Despite its name, JavaScript has almost nothing in common with the Java language. It was created in 1995 for the Netscape browser and was named "JavaScript" mainly for marketing reasons. Today it is the language behind almost every interactive site you use in Cameroon: the balance check page of MTN Mobile Money, the e-government portals, and the results pages of the GCE Board all rely on JavaScript.

Inserting JavaScript into a page. There are two standard ways:

- **Internal script**: the code is written directly in the HTML file between the tags `<script>` and `</script>`, usually placed just before `</body>` so that the page content loads first.
- **External script**: the code is saved in a separate file, for example `script.js`, and linked with `<script src="script.js"></script>`. This is the recommended practice for larger projects because the same script can be reused by several pages and the browser can cache it.

WARNING **Placement of `<script>`**

If the script tries to access HTML elements that have not yet been parsed (for example, a script in `<head>` that reads a `<div>` in `<body>`), it will fail. Placing the script just before `</body>` or using the `defer` attribute (`<script src="script.js" defer></script>`) guarantees that the DOM is ready.

A first script. Three instructions are essential for a beginner:

- `alert("Hello")` displays a modal pop-up message box;
- `document.write("Hello")` writes text directly into the page (mainly used for simple tests; it overwrites the whole page if called after the page has loaded);
- `console.log("Hello")` writes a message in the browser's developer console (opened with the F12 key), which is invisible to visitors and therefore ideal for debugging, i.e. finding errors.

METHOD **Writing and testing a first JavaScript program**

1. Open a text editor (Notepad++, VS Code) and create a file `index.html` containing the basic HTML skeleton.
2. Just before `</body>`, add `<script>` then your instructions, for example `alert("Welcome to my page");`, then `</script>`.
3. Save the file and open it with a browser (double-click).
4. Observe the result. If nothing happens, press F12 to open the console and read the error message: it indicates the line where the problem lies.
5. Correct, save, and refresh the page (F5) to test again.

WARNING **JavaScript is case-sensitive**

JavaScript distinguishes uppercase from lowercase letters. Writing `Document.write("Hi")` or `Alert("Hi")` produces an error, because the correct names are `document.write` and `alert`. Similarly, a variable declared as `average` is different from a variable called `Average`. Most "my script does nothing" problems of beginners come from a wrong capital letter.

3.2 Variables and Data Types

A **variable** is a named memory location in which the program stores a value that may change during execution. In modern JavaScript a variable is declared with one of two keywords:

- `let x = 15;` declares a variable whose value can be modified (block-scoped, recommended);
- `const PI = 3.14;` declares a constant whose value cannot be reassigned (also block-scoped).

The keyword `var` is the old way of declaring variables (function-scoped); it is still found in many textbooks and past papers, but `let` and `const` are preferred in modern code.

Naming rules. A variable name (identifier) must begin with a letter, an underscore _ or a dollar sign `$`; it may then contain letters, digits, _ and `$`; it must not be a reserved word (`if`, `for`, `function`…) and must not contain spaces or hyphens. Valid: `mark1`, `total_sum`, `average`. Invalid: `1mark`, `my mark`, `total-sum`.

Data types. JavaScript has primitive types and objects. The three basic primitive types you must master are:

- **number**: numeric values, integers or decimals, e.g. `let age = 17;` or `let avg = 12.75;` (note the decimal point);
- **string**: text, always written between single or double quotes, e.g. `let name = "Amina";`;

- **boolean**: a logical value, only `true` or `false`, e.g. `let admitted = true;`.

Two other primitive types exist: `undefined` (variable declared but not assigned) and `null` (intentional absence of value).

EXEMPLE **Declaring and initialising variables**

```
let studentName = "Ngono";    // string
let mathMark = 14;            // number (integer)
let physicsMark = 13.5;       // number (decimal)
let isAdmitted = true;        // boolean
const MAX\_MARK = 20;          // constant
```

3.3 Operators

Operators combine values to produce new values.

Category	Operators	Example / result
Arithmetic	`+` `-` `*` `/` `%` (remainder)	`14 % 4` gives `2`
Comparison	`==` (equal), `===` (strictly equal), `!=` (different), `<`, `>`, `<=`, `>=`	`10 >= 10` is `true`
Logical	`&&` (AND), `||` (OR), `!` (NOT)	`true && false` is `false`
Assignment	`=`, `+=`, `-=`, `*=`, `/=`	`x += 3` adds 3 to `x`

The difference between `==` and `===` is that `==` compares values after converting types (so `"5" == 5` is `true`), while `===` also requires the same type (so `"5" === 5` is `false`). At this level, prefer `===` and `!==` to avoid surprising type coercion.

WARNING **Assignment `=` versus comparison `==` / `===`**

The single equals sign `=` is the assignment operator: `avg = 10` stores 10 in `avg`. To test equality you must write `avg == 10` or `avg === 10`. Writing `if (avg = 10)` does not compare anything: it puts 10 into `avg` and the condition is then treated as true, so the program always takes the wrong branch. This is one of the most frequent errors at the GCE.

Reading user input. The instruction `prompt("Enter a mark:")` opens a small dialog box and returns what the user typed — always as a string. To calculate with it, convert it: `parseInt("12")` gives the integer `12`, and `parseFloat("12.5")` gives the decimal number `12.5`. Forgetting this conversion is a classic trap: `"2" + "3"` gives the string `"23"` (text concatenation), whereas `parseInt("2") + parseInt("3")` gives the number `5`.

EXEMPLE **Input, conversion and calculation**

```
let a = prompt("First mark:");       // user types "12" -> string "12"
let b = prompt("Second mark:");      // user types "14" -> string "14"
let sum = parseInt(a) + parseInt(b); // 12 + 14 = 26 (number)
alert("Sum = " + sum);               // displays "Sum = 26"
```

3.4 Conditional Structures: if, if…else, switch

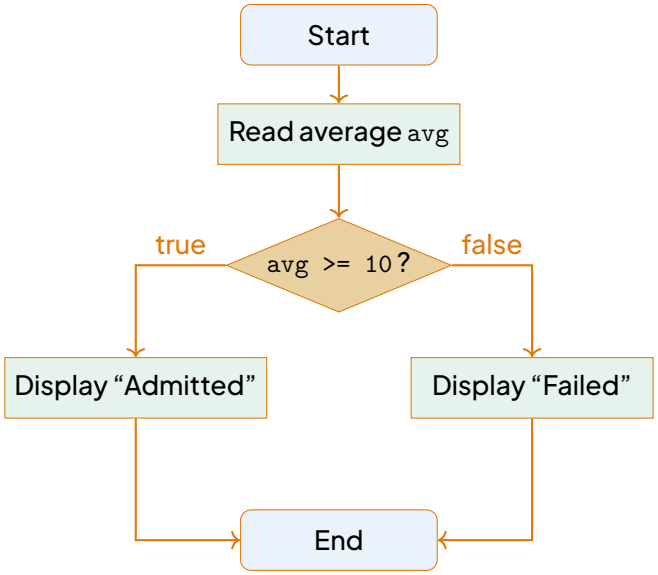
Programs become intelligent when they can choose between several actions according to a condition.

The `if` statement. The syntax is: `if (condition) { instructions }`. The block between braces executes only when the condition evaluates to `true`. With `if … else` we provide an alternative block executed when the condition is `false`; with `else if` we chain several tests. Example: deciding whether a GCE candidate is admitted.

EXEMPLE **Admission decision**

```
let avg = parseFloat(prompt("Average?"));
if (avg >= 10) {
    alert("Admitted");
} else {
    alert("Failed");
}
```

If the user enters 12.5, the condition `12.5 >= 10` is true, so the message “Admitted” appears. If the user enters 8, the `else` branch runs and “Failed” appears.



Flowchart of an `if…else` structure deciding the admission of a GCE candidate (average ≥ 10).

The `switch` statement. When one variable must be compared with several fixed values, `switch` is clearer than a long chain of `else if`. The syntax:

```
switch (expression) {
    case value1:
        instructions;
        break;
    case value2:
        instructions;
        break;
    default:
        instructions;
}
```

Each `case` tests one value; `break` stops the execution inside the switch; `default` runs when no case matches. Forgetting `break` makes the following cases execute too (“fall-through”) — another classic mistake.

EXEMPLE **Grade to comment with switch**

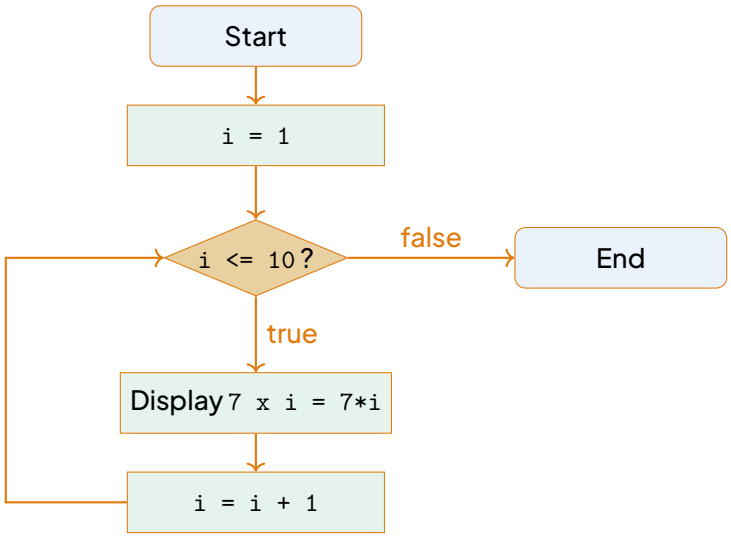
```
let grade = prompt("Grade (A, B, C, D, E):");
switch (grade) {
    case "A": alert("Excellent"); break;
    case "B": alert("Good"); break;
    case "C": alert("Fair"); break;
    case "D": alert("Pass"); break;
    case "E": alert("Fail"); break;
```

```
    default: alert("Invalid grade");
}
```

3.5 Loops: for and while

A **loop** repeats a block of instructions. Instead of writing `document.write` ten times to print the multiplication table of 7, we write it once inside a loop.

The `for` loop is used when the number of repetitions is known in advance: `for (initialisation; condition; increment) { instructions }`. Example: `for (let i = 1; i <= 10; i++) { document.write x " + i + " = " + (7*i) + "<br>"); }`. Here `i` starts at 1, the body repeats while `i <= 10`, and `i++` increases `i` by 1 after each pass.



Flowchart of a `for` loop printing the multiplication table of 7, for `i` from 1 to 10.

The `while` loop repeats a block as long as a condition remains true; it is used when the number of repetitions is not known in advance: `while (condition) { instructions }`. The condition is tested before each pass, so the body may execute zero times. You must ensure that something inside the loop eventually makes the condition false, otherwise you get an infinite loop and the page freezes.

EXEMPLE **Sum of positive numbers with while**

```
let sum = 0;
let value = parseFloat(prompt("Enter a positive number (negative to stop):"));
while (value >= 0) {
    sum = sum + value;
    value = parseFloat(prompt("Enter a positive number (negative to stop):"));
```

```
}
alert("Sum = " + sum);
```

The loop stops as soon as the user enters a negative number.

WORKED EXAMPLE 3 **Sum of marks with a for loop**

Statement. Write a script that asks for the marks of 3 subjects and displays their sum and average. Predict the output if the user enters 12, 15 and 9.

SOLUTION

Solution.

```
let sum = 0;
for (let i = 1; i <= 3; i++) {
    let m = parseFloat(prompt("Mark " + i));
    sum = sum + m;
}
alert("Sum = " + sum + ", Average = " + (sum/3));
```

Trace: initially `sum = 0`. Pass 1: `i = 1`, `m = 12`, `sum = 12`. Pass 2: `i = 2`, `m = 15`, `sum = 27`. Pass 3: `i = 3`, `m = 9`, `sum = 36`. Then `i` becomes 4, the condition `4 <= 3` is false, the loop stops. The alert displays "Sum = 36, Average = 12".

KEY POINTS **Exam tip: tracing a script**

A classic GCE question gives a short script and asks you to predict its output. Method: draw a small table with one column per variable and one row per pass through the loop; update the values line by line exactly as the computer would; then write down what each `alert` or `document.write` displays. Never guess — trace. Pay special attention to the initial value of the counter, to whether the condition uses < or <=, and to the order of the instructions inside the loop.

3.6 Functions: Avoiding Repetition

Suppose three different buttons of your page must each compute an average. Copying the same five lines three times is bad practice: if you later correct a mistake, you must correct it in three places. A **function** packages a block of instructions under a name, so it can be called (executed) as many times as needed.

Declaring and calling. A function is declared once: `function average(a, b) { let r = (a + b) / 2; return r; }`. Here `a` and `b` are parameters: placeholders that receive the values given

at the call. The keyword `return` sends the result back to the caller. The function does nothing until it is called: `let m = average(12, 16);` executes the body with `a = 12`, `b = 16`, and stores the returned value `14` in `m`. A function without a `return` statement returns `undefined`.

EXEMPLE **A reusable function**

```
function square(x) {
    return x * x;
}
document.write(square(5));  // displays 25
document.write(square(9));  // displays 81
```

The same code serves both calculations: this is the power of functions — write once, use many times.

EXEMPLE **Function with multiple parameters and no return**

```
function greet(name, age) {
    alert("Hello " + name + ", you are " + age + " years old.");
}
greet("Amina", 17);  // calls the function
```

EXERCISE 3 **Practice**

1. Write a script that reads an integer with `prompt()` and displays “Even” or “Odd” using the operator `%`.
2. Trace the following script and give the exact output: `let s = 0; for (let i = 1; i < 5; i++) { s = s + i; } alert(s);`
3. Write a function `maxOfTwo(a, b)` that returns the larger of its two parameters, then call it with the values 14 and 9.
4. Write a `while` loop that asks the user for numbers and accumulates their sum until the user enters 0, then displays the sum.

SOLUTION **Exercise n°2**

Trace: `i = 1`: `s = 1`; `i = 2`: `s = 3`; `i = 3`: `s = 6`; `i = 4`: `s = 10`. Then `i = 5` and the condi-

tion `5 < 5` is false, so the loop stops. The alert displays `10`. Note the strict inequality: the pass `i = 5` never happens.

SOLUTION **Exercise n°3**

```
function maxOfTwo(a, b) {
    if (a > b) return a;
    else return b;
}
alert(maxOfTwo(14, 9));  // displays 14
```

SOLUTION **Exercise n°4**

```
let sum = 0;
let n = parseFloat(prompt("Enter a number (0 to stop):"));
while (n !== 0) {
    sum = sum + n;
    n = parseFloat(prompt("Enter a number (0 to stop):"));
}
alert("Sum = " + sum);
```

With variables, operators, conditions, loops and functions, you now master the fundamental grammar of JavaScript. The next section will put these tools to work on the page itself: reading and modifying HTML elements through the DOM, reacting to events such as clicks, and validating forms before they are submitted.

4 INTERACTIVE PAGES: DOM MANIPULATION, EVENTS AND FORM VALIDATION

In the previous section we learnt the grammar of JavaScript: variables, operators, conditions and loops. But a language alone does not make a page alive. A script becomes useful only when it can **see** the page and **change** it: react when a visitor clicks a button, read what was typed into a form, refuse to send incomplete data. That is exactly what this final section is about. By the end, you will be able to build a complete interactive registration form — the kind of page used every day in Cameroon for school enrolment, mobile money registration or examination candidate registration on the GCE Board portal.

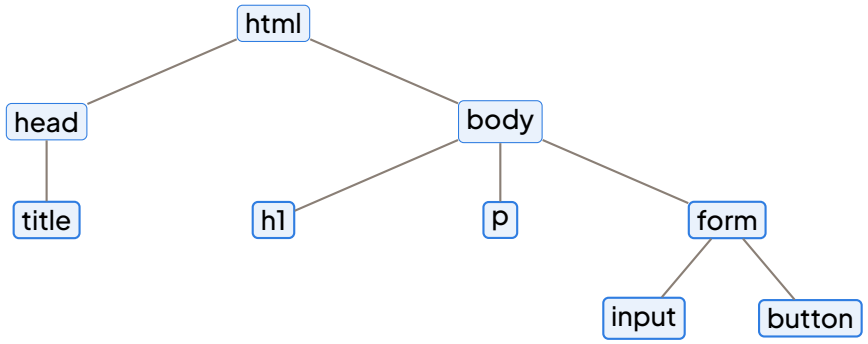
4.1 The Document Object Model (DOM)

EXPERIMENT **Observing the page as a family tree**

Consider this tiny page: a heading, a paragraph, and a form containing one text field and one button. If you were asked to describe the page to someone over the phone, you would naturally say: "the page has a head and a body; inside the body there is a heading, a paragraph and a form; inside the form there is an input and a button". You have just described a tree. JavaScript sees every web page exactly this way.

DEFINITION **The Document Object Model**

The **DOM** (Document Object Model) is the representation of a web page as a tree of objects created by the browser when it loads the HTML document. Every HTML element (and even the text inside elements) becomes a node of the tree, and JavaScript can read, modify, add or delete any node. The root object is called `document`.



The DOM tree of a small HTML page: `html` is the root; `head` and `body` are its children; the `form` itself has two children, `input` and `button`.

Because the page is a tree of objects, each element carries **properties** (its text, its colour, its value) that JavaScript can consult and change. The whole art of interactivity reduces to three steps: **find** the node, **read or change** it, and decide **when** to do it (in response to an event).

4.2 Selecting Elements and Modifying the Page

Before changing anything, the script must first grab the element. JavaScript offers several selection methods:

Method	What it returns
`document.getElementById('b1')`	the single element whose `id` is `b1` (or `null` if none)
`document.getElementsByClassName('note')`	a list of all elements of class `note`
`document.querySelector('p')`	the first element matching the CSS selector (here the first paragraph)

WARNING **Script placed too early: getElementById returns null**

The browser reads the page from top to bottom. If your script is placed in the `head` and calls `getElementById('b1')` before the button `b1` has been created lower in the page, the method returns `null` and the script fails. Good habit: place the `script` block just before the closing `body` tag, so that all elements already exist when the script runs.

Once an element is selected, three properties do most of the work:

- `element.textContent`: reads or replaces the **text** of the element, e.g. `document.getElementById('msg').textContent = 'Welcome!';`.
- `element.innerHTML`: reads or replaces the **HTML content** (tags included), e.g. `innerHTML = '<b>Admitted</b>';` displays bold text.
- `element.style.property`: changes a CSS property directly, e.g. `document.body.style.backgroundColor = 'yellow';`. Note that CSS names with hyphens become camelCase: `background-color` becomes `backgroundColor`.

Reading what a user typed into a form field is done with the `value` property: `document.getElementById('phone').value` gives the content of the field whose `id` is `phone`.

WARNING **'2' + '3' gives '23', not 5**

The `value` of a form field is always a string of characters, even if the user typed digits. Writing `'2' + '3'` performs concatenation and produces `'23'`. To compute, convert first with `Number(...)` or `parseInt(...)`: `Number(a) + Number(b)` gives 5. This is one of the most frequent errors in GCE practical scripts.

4.3 Events: Making the Page React

DEFINITION **Event**

An **event** is a signal produced by the browser when something happens on the page: a click, a mouse movement, a key pressed, a form submitted. A JavaScript function attached to an event and executed automatically when it occurs is called an event handler.

The most common events at this level are:

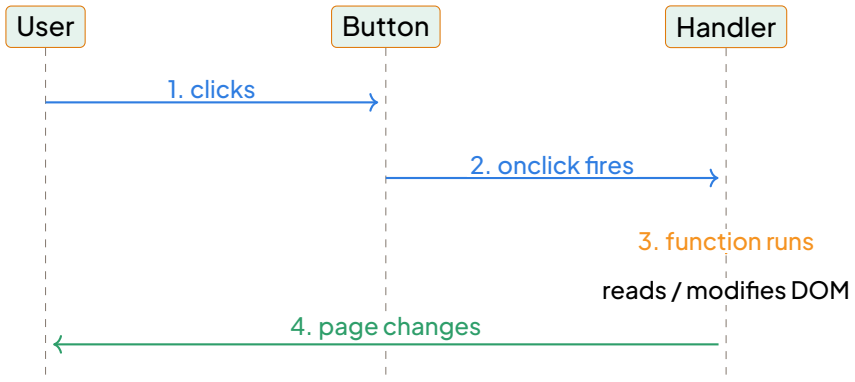
Event	Fired when…
`onclick`	the user clicks an element
`onmouseover`	the mouse pointer moves onto an element
`onchange`	the value of a field changes and loses focus
`onsubmit`	a form is about to be submitted
`onkeypress`	a key is pressed while a field has focus

A handler can be attached in two ways:

- **HTML attribute:** `<button onclick="change()">Click</button>` — simple and common at GCE level.
- **addEventListener:** `document.getElementById('b1').addEventListener('click', change);` — separates HTML from JavaScript; note the event name is written without "on".

METHOD **Making an element react to a click**

1. Select the element, e.g. give it an `id` and use `getElementById`.
2. Attach the event: write `onclick="myFunction()"` in the tag, or use `addEventListener('click', myFunction)`.
3. Write the handler function `myFunction()` containing the actions to perform (change text, colour, visibility…).



Sequence of an interaction: the user clicks, the event fires, the handler function runs and modifies the DOM, and the page updates instantly without reloading.

EXEMPLE **A button that changes the background colour**

HTML: `<button id="b1" onclick="paint()">Paint</button>`.

Script (placed before `</body>`): `function paint() { document.body.style.backgroundColor = 'lightblue'; }`.

To show/hide a paragraph instead, toggle its display: `var p = document.getElementById('para'); p.style.display = (p.style.display === 'none') ? 'block' : 'none';`.

4.4 Form Validation with JavaScript

A form that sends empty or wrong data wastes time and server resources — think of a candidate registering for the GCE with a missing phone number. **Client-side validation** checks the data in the browser, before submission.

METHOD **Validating a form before submission with onsubmit**

1. Attach the handler to the form: `<form onsubmit="return check()">`.
2. In `check()`, read each field with `document.getElementById('…').value`.
3. Test each rule: field not empty (`value === ''`), phone has 9 digits (`value.length === 9`), two passwords identical (`p1 === p2`).
4. If a rule fails: display an error message and `return false;` — this cancels the submission. If all rules pass, `return true;` and the form is sent.

WORKED EXAMPLE 4 **Validating a phone number and two matching passwords**

A form contains fields `phone` (a Cameroonian mobile number, 9 digits, e.g. 6XXXXXXXX), `pwd` and `pwd2`, plus a paragraph `err` for messages. Write the validation function.

SOLUTION

Solution. The function reads the three values, tests them in order, writes a message and

returns `false` on the first failure:

```
function check() {
  var ph = document.getElementById('phone').value;
  var p1 = document.getElementById('pwd').value;
  var p2 = document.getElementById('pwd2').value;
  var e = document.getElementById('err');
  if (ph.length !== 9) { e.textContent = 'Phone must have 9 digits'; return false; }
  if (p1 === '') { e.textContent = 'Password required'; return false; }
  if (p1 !== p2) { e.textContent = 'Passwords do not match'; return false; }
  return true;
}
```

The form tag is `<form onsubmit="return check()">`. The keyword `return` before `check()` is essential: it transmits the `false` to the browser, which then blocks the submission.

4.5 Guided Project: An Interactive School Registration Form

EXPERIMENT **Building the form step by step**

We build a mini registration page for "GBHS OnBuch" with live validation and a welcome message.

1. HTML skeleton: a form with fields `fullname`, `phone`, a button `reg`, a paragraph `msg` for feedback.
2. Live check on the phone field: attach `onkeypress` or `onchange` so that as soon as the field loses focus, a handler colours the field border green (9 digits) or red (otherwise) using `element.style.borderColor`.
3. Submission check: `<form onsubmit="return register()">`; the function refuses empty names and bad phone numbers with `return false`.
4. Welcome message: when everything is valid, the handler writes `'Welcome ' + name + ', registration received!'` into `msg` using `textContent`, and changes the page background to a soft green.

The complete logic is exactly the combination of the methods above: select, read `.value`, test, modify the DOM, return `true` or `false`.

KEY POINTS **Key points**

- The DOM is the page seen as a tree of objects; `document` is its root.
- Select with `getElementById`, `getElementsByClassName` or `querySelector`; modify with `textContent`, `innerHTML` and `style`.
- An event (click, submit, keypress…) triggers a handler function; `return false` in `onsubmit` blocks the submission.
- Field values are strings: convert before calculating.

DID YOU KNOW? **Libraries and frameworks — awareness only**

Professional developers rarely write all this by hand: libraries such as jQuery simplify selection and events (`$('#b1').click(...)`) and frameworks such as React or Angular build whole applications from reusable components. They all still rely on the DOM and events you have just learnt. This is for your culture only — not examinable in depth at GCE level.

WARNING **Exam tip — GCE practical style**

Practical papers often show a half-written script and ask you to complete it: fill in the `getElementById` call, write the condition that tests an empty field, or add `return false`. Train yourself to read the HTML first, spot the `id` of each element, then trace the script line by line.

EXERCISE 4 **Show and hide**

Write the HTML and JavaScript for a page with a button and a paragraph of advice. Each click on the button hides the paragraph if it is visible, and shows it again if it is hidden.

SOLUTION **Exercise 1**

HTML: `<button onclick="toggle()">Show/Hide</button>` and `<p id="adv">Revise every day!</p>`. Script: `function toggle() { var p = document.getElementById('adv'); if (p.style.display === 'none') { p.style.display = 'block'; } else { p.style.display = 'none'; } }`. The script is placed just before `</body>` so that the paragraph exists when the handler runs.

5 INTEGRATION ACTIVITY

5.1 Aims of the activity

This integration activity closes the chapter Apply common technologies for web development. It mobilises, in a single realistic project, all the competences developed in Lessons 47 and 48. By the end of the activity, each group of learners should be able to:

- write an **external CSS stylesheet** and link it correctly to an HTML page using the `link` element;
- use **selectors** (element, class, id, descendant) to apply styles precisely;
- style text, colours, backgrounds, tables (borders, alternating row colours) and lay out elements using the **box model** (margin, border, padding, content);
- declare and use JavaScript **variables**, **functions**, **conditions** (`if`/`else`) and simple loops;
- access and modify page content through the **DOM** (`getElementById`, `value`, `innerHTML`);
- react to **events** (`onclick`, `onchange`, `oninput`);
- **validate a form** before submission (non-empty fields, 9-digit Cameroonian phone number);
- work in a team, present a product and justify technical choices, as expected at the GCE Advanced Level.

KEY POINTS **Competence targeted**

The learner integrates CSS styling and JavaScript interactivity to design, implement, test and present a complete, attractive and functional one-page website for a local business.

5.2 Situation

Mama Ngo's Catering – Buea.

Mrs. Ngo Bekono runs a small catering business near Molyko, Buea. She prepares traditional dishes for students, staff of the University of Buea and families celebrating events. Until now, customers place orders by phone call or WhatsApp message, and Mrs. Ngo writes everything in a notebook. She frequently makes mistakes: she misreads phone numbers, forgets quantities, and customers often quarrel about the total price because it is computed by hand.

Her nephew, a student in Upper Sixth, has convinced her that a simple one-page website opened on any phone browser would solve these problems. He has already written the basic HTML structure, but the page is ugly (plain black text on white background, no layout) and completely static (no calculation, no verification). Mrs. Ngo hires **your group** to finish the work.

Data provided by Mrs. Ngo.

Her price list is fixed and must appear in a table on the page:

Code	Dish (service)	Unit price (FCFA)
P1	Eru with water fufu (plate)	1500
P2	Ndolé with miondo (plate)	2000
P3	Poulet DG (plate)	2500
P4	Fried fish with plantain (plate)	1800
P5	Full event catering (per guest)	3500

Her requirements (the contract).

1. A **header** displaying the business name "Mama Ngo's Catering – Buea" and her slogan "The taste of home", using her favourite colours: deep green background, white text, gold underline. All styling must be in an **external CSS file** named `style.css`.
2. The **price table** must have visible borders, a coloured header row, and **alternating row colours** (zebra striping) so that customers read prices easily.
3. An **order form** with four fields: customer name, phone number, product (a drop-down list of the five dishes) and quantity.
4. **JavaScript validation**: when the customer clicks "Send order", the script must refuse the order if any field is empty or if the phone number does not contain **exactly 9 digits** (Cameroonian mobile numbers such as `690123456`), and display a clear red error message.
5. **Automatic total**: as soon as the quantity or the product changes, the page must display the **total cost in FCFA** (unit price × quantity) without reloading the page.
6. **Confirmation**: after a valid order, a personalised message such as "Thank you Amina! Your order of 3 plate(s) of Poulet DG, total 7500 FCFA, has been received. We will call you on 690123456." must appear in green.

Your group must produce the three files (`index.html`, `style.css`, `script.js`), test the page in a browser, and present it to the class, justifying every important CSS and JavaScript choice.

5.3 Tasks

Task 1 – Structure and integration of CSS.

1. Recall the three ways of integrating CSS into an HTML page. State which one Mrs. Ngo imposes and give **two advantages** of this method.
2. Write the HTML line that links `style.css` to `index.html`, and state precisely where it must be placed.

Task 2 – Styling the page.

3. Write the CSS rules for the header: deep green background (for example `#1B5E20`), white centred text, and a gold underline (`#FFD700`) under the slogan. Explain which selector you use and why.
4. Using the **box model**, give the page content a comfortable spacing: explain the difference between `margin` and `padding`, then write a rule that gives the main content an inner spacing of `20px` and centres it with a maximum width of `800px`.

5. Style the price table: collapsed borders, a green header row with white text, and alternating row colours (white / light grey `#F2F2F2`) using the `:nth-child(even)` pseudo-class. Explain in one sentence how `:nth-child(even)` works.

Task 3 – JavaScript interactivity.

6. Write a JavaScript function `computeTotal()` that reads the unit price of the selected product and the quantity from the DOM, computes the total, and displays it in a paragraph with `id="total"`. State on which **events** this function must be attached so that the total updates immediately.

7. Write a function `validateOrder()` that: (a) checks that no field is empty; (b) checks that the phone number contains exactly 9 digits; (c) displays a red error message or, if everything is valid, calls the confirmation function. Explain why a regular expression such as `/{}$\backslash$d{9}$/` is more reliable than simply checking the length.

8. Write the function `showThanks()` that builds the personalised message using the values entered and displays it in green. Which DOM property do you use to inject the text into the page?

9. Mrs. Ngo notices that typing letters in the quantity field breaks the total. Propose and justify **one HTML solution** and **one JavaScript solution** to this problem.

Task 4 – Presentation and justification.

10. Prepare a 5-minute presentation: demonstrate the page live (valid order, then an order with a bad phone number), and justify at least three technical choices (choice of selectors, external CSS, event handling, validation strategy).

WARNING **Common traps to avoid**

Linking the CSS file with a wrong path or forgetting `rel="stylesheet"`; confusing `id` and `class` selectors; reading a form value before the user has typed (script placed before the HTML elements without waiting for the page to load); comparing with `=` instead of `==` or `===`; forgetting that `value` returns a string, so `"2" + "3"` gives `"23"` and not `5` – convert with `Number()` or `parseInt()`.

5.4 Model answer

Task 1. The three integration methods are: inline CSS (attribute `style` on an element), internal CSS (a `style` block in the `head`) and external CSS (a separate `.css` file). Mrs. Ngo imposes the **external** method. Advantages: (1) the same stylesheet can style several pages, so changes are made once; (2) it separates content (HTML) from presentation (CSS), making the code easier to read and maintain. The link is placed inside the `head` element:

```
<link rel="stylesheet" href="style.css">
```

Task 2 – Question 3. Using an `id` selector because the header is unique:

```
#header { background-color: #1B5E20; color: white; text-align: center; padding:
                                    20px; }
#slogan { text-decoration: underline; text-decoration-color: #FFD700; font-style:
                                    italic; }
```

Question 4. `padding` is the space inside the box, between the content and the border; `margin` is the space outside the border, separating the box from its neighbours. Centring rule:

```
main { max-width: 800px; margin: 0 auto; padding: 20px; }
```

`margin: 0 auto` gives a zero vertical margin and automatically equal left/right margins, which centres the block horizontally.

Question 5. Table styling:

```
table { border-collapse: collapse; width: 100%; }
th, td { border: 1px solid #999; padding: 8px; }
th { background-color: #1B5E20; color: white; }
tr:nth-child(even) { background-color: #F2F2F2; }
```

`:nth-child(even)` selects every row whose position among its siblings is a multiple of 2 (rows 2, 4, 6, …), producing the zebra effect.

Task 3 – Question 6. The function `computeTotal()`:

```
                        function computeTotal() {
    \quad var price = Number(document.getElementById("product").value);
      \quad var qty = Number(document.getElementById("qty").value);
                    \quad var total = price * qty;
\quad document.getElementById("total").innerHTML = "Total: " + total + " FCFA";
                                    }
```

The drop-down list stores each price as the `value` of its `option` (e.g. `<option value="2500">Poulet DG -- 2500 FCFA</option>`). The function is attached to the events `onchange` of the product list and `oninput` of the quantity field, so the total updates immediately, without any button click or page reload.

Question 7. Validation:

```
                        function validateOrder() {
    \quad var name = document.getElementById("name").value.trim();
   \quad var phone = document.getElementById("phone").value.trim();
        \quad var qty = document.getElementById("qty").value;
          \quad var err = document.getElementById("error");
       \quad if (name === "" || phone === "" || qty === "") {
\quad\quad err.innerHTML = "Please fill in all fields."; return; }
          \quad if (!/{}$\backslash$d{9}$/.test(phone)) {
\quad\quad err.innerHTML = "Phone number must contain exactly 9 digits."; return; }
          \quad err.innerHTML = ""; showThanks(name, phone); }
```

The regular expression `/{}$\backslash$d{9}$/` is more reliable than a length test because it simultaneously imposes that the entry starts (`{}`) and ends (`$`) with exactly nine digits (`$\backslash$d{9}`): an input like `69012AB56`, which has length 9 but contains letters, is rejected.

Question 8. The confirmation function injects the message with the DOM property `innerHTML`:

```
                    function showThanks(name, phone) {
                \quad var msg = document.getElementById("thanks");
                    \quad msg.style.color = "green";
\quad msg.innerHTML = "Thank you " + name + "! Your order has been received. We
                    will call you on " + phone + "."; }
```

Question 9. HTML solution: use `<input type="number" min="1">` so the browser itself blocks non-numeric input and values below 1. JavaScript solution: test with `isNaN(qty)` or force a minimum in the code (`if (qty < 1) qty = 1;`). Using both together is best practice: HTML gives a first barrier, JavaScript guarantees the check even on old browsers.

Task 4. A good presentation demonstrates: a valid order (name, `690123456`, 3 plates of Poulet DG → total 3 × 2500 = 7500 FCFA, green thank-you message), then a rejected order (phone `67890123`, only 8 digits → red error). Justifications expected: external CSS for reusability and maintainability; `id` selectors for unique elements and `class` selectors for repeated styles; `oninput`/`onchange` events for instant feedback; regular-expression validation for robustness; conversion with `Number()` to avoid string concatenation bugs.

EXPERIMENT **Evaluation grid suggested for the teacher**

External CSS correctly linked and used (2 pts); header and table styling with zebra rows (3 pts); box model applied (2 pts); total computed live (3 pts); validation of empty fields and 9-digit phone (4 pts); personalised confirmation (2 pts); quality of presentation and justification (4 pts). Total: 20 pts.

6 METHODS AND WORKED EXAMPLES

7 INTRODUCTION TO CSS: SYNTAX, SELECTORS AND INTEGRATION

CSS (Cascading Style Sheets) is the language that describes how HTML elements are presented on screen. It separates **structure** (HTML) from **presentation** (layout, colours, fonts), making sites easier to maintain and restyle. In this section we learn the anatomy of a CSS rule, the three ways to attach styles to a page, and how the browser resolves conflicts between competing rules.

DID YOU KNOW? **CSS in Cameroon**

Many Cameroonian government portals (e.g. `www.minesec.gov.cm`, `www.antic.cm`) use external stylesheets to keep a consistent visual identity across dozens of pages. When the Ministry updates its colour palette, only one `.css` file changes.

7.1 Method 1: Writing and Integrating CSS Rules

METHOD **Writing a correct CSS rule and linking it to a page**

A CSS **rule** consists of a **selector** and a **declaration block**. To apply CSS to a web page, follow these steps:

1. Identify the HTML element(s) you want to style (tag, class or id).
2. Choose the correct selector: `p` selects all paragraphs, `.menu` selects every element with `class="menu"`, `#logo` selects the single element with `id="logo"`.
3. Write the declaration block in curly braces; each declaration is `property: value;` — never forget the colon and the semicolon.
4. Choose the integration method:
   - Inline: `style="..."` inside a tag (quick tests only).
   - Internal: a `<style>` block in the `<head>` (single-page styles).
   - External: a separate `.css` file linked with `<link rel="stylesheet" href="style.css">` in the `<head>`. For any serious project, prefer the external stylesheet: one file styles the whole site.
5. Open the page in a browser and check the result; if a rule is ignored, check spelling of

the property and the selector first.

Reflex: one selector, one pair of braces, one semicolon per declaration. An external file never contains `<style>` tags.

KEY POINTS **Anatomy of a CSS rule**

Part	Example
Selector	`#titre`
Declaration block	`{ color: blue; text-align: center; }`
Property	`color`
Value	`blue`

WARNING **Common syntax errors**

- Missing semicolon after a declaration (the last one before `}` is optional but good practice).
- Misspelled property names (e.g. `font-colour` instead of `color`).
- Using `=` instead of `:` between property and value.
- Putting HTML tags inside an external `.css` file.

WORKED EXAMPLE 5 **Styling the page of a Douala cybercafé**

A student in Bonabéri creates a page `index.html` for a cybercafé. The page contains a heading `<h1 id="titre">Douala Net Café</h1>`, several paragraphs of class `info`, and a list of services.

1. Write the CSS rule that colours the main heading in blue and centres it.
2. Write the rule that gives all `info` paragraphs a font size of 14 pixels and grey colour.
3. Show the line that links an external file `style.css` to the HTML page, and state where it is placed.

SOLUTION

1. Rule for the heading. The heading has an `id`, so we use the `#` selector. An id is unique in a

page, which is exactly what we want for a main title:

```
#titre { color: blue; text-align: center; }
```

The selector `#titre` targets the element whose `id` is `titre`; `color` sets the text colour and `text-align: center` centres the text horizontally.
2. Rule for the paragraphs. Several paragraphs share `class="info"`, so we use the class selector, written with a dot:

```
.info { font-size: 14px; color: gray; }
```

Every element carrying the class `info` receives both declarations. Note the unit `px` after the number: a length without a unit is invalid (except the value `0`).
3. Linking the external stylesheet. The link is placed inside the `<head>` section of the HTML document, never in the `<body>`:

```
<link rel="stylesheet" href="style.css">
```

The attribute `rel="stylesheet"` declares the relationship, and `href` gives the path to the file. Since both files are in the same folder, the file name alone is enough. From now on, any change in `style.css` is reflected on every page that links it.

7.2 Method 2: Choosing the Right Selector and Managing Conflicts

METHOD **Selectors, specificity and the cascade**

When several rules target the same element, the browser must decide which one wins.

1. List all rules that match the element.
2. Apply the **specificity** order, from weakest to strongest:
   - Type selector (`p`, `h1`, `div`)
   - Class selector (`.info`, `.menu`) and attribute selectors (`[type="text"]`)
   - ID selector (`#titre`, `#logo`)
   - Inline style (`style="..."`)
3. If two rules have equal specificity, the one that appears last in the stylesheet wins (this is the cascade, the “C” in CSS).
4. Use **descendant selectors** (`div p`) to target elements inside a given container, **child selectors** (`ul > li`) for direct children, and group selectors with a comma (`h1, h2, h3`) to share declarations.
5. Pseudo-classes (`:hover`, `:focus`, `:nth-child`) and pseudo-elements (`::before`, `::after`) have the same specificity as classes.

Reflex: before saying “my CSS does not work”, ask: is a more specific rule overriding mine?

KEY POINTS **Specificity hierarchy (simplified)**

inline style > ID selector > class/attribute/pseudo-class > type selector > universal selector (*)

The `!important` flag overrides everything but should be avoided in maintainable code.

WORKED EXAMPLE 6 **Predicting the colour of a paragraph**

A page contains:

```
<p id="actu" class="info">Match at 4 pm in Yaoundé.</p>
```

and the stylesheet contains, in this order:

```
p { color: black; }
.info { color: green; }
#actu { color: red; }
```

1. What colour is displayed for the paragraph? Justify.
2. The class rule is moved after the id rule. Does the colour change?

SOLUTION

1. Displayed colour: red. All three selectors match the paragraph: it is a `p`, it has the class `info`, and it has the id `actu`. We therefore compare specificity. The type selector `p` is the weakest; the class selector `.info` is stronger; the id selector `#actu` is the strongest of the three. The id rule wins, so the text appears in red.
2. Moving the class rule: no change. The rule “the last one wins” applies only between rules of equal specificity. Here the three rules have different specificity levels, so the id rule still wins wherever it is placed in the file. The paragraph remains red. This is a classic GCE trap: order matters only as a tie-breaker, never to beat a more specific selector.

8 STYLING WITH CSS: TEXT, COLOURS, BACKGROUNDS AND THE BOX MODEL

8.1 Method 3: Styling Text, Colours and Backgrounds

METHOD **Essential text, colour and background properties**

1. Font and text: `font-family`, `font-size`, `font-weight`, `font-style`, `line-height`, `text-align`, `text-decoration`, `text-transform`, `letter-spacing`.
2. Colours: named colours (`red`), hexadecimal (`#ff0000`, `#f00`), RGB (`rgb(255,0,0)`), RGBA (`rgba(255,0,0,0.5)`), HSL (`hsl(0,100%,50%)`).
3. Backgrounds: `background-color`, `background-image`, `background-repeat`, `background-position`, `background-size`, `background-attachment`. The shorthand `background` sets all at once.
4. Always provide a fallback `font-family` list ending with a generic family (`serif`, `sans-serif`, `monospace`).

Reflex: use relative units (`rem`, `em`, `%`) for font sizes to respect user preferences and accessibility.

EXEMPLE **A styled notice for a Buea school**

```
.notice { font-family: "Segoe UI", Arial, sans-serif; font-size: 1.1rem;
line-height: 1.5; color: #333; background-color: #f0f8ff; padding: 1rem;
border-left: 4px solid #0066cc; }
```

8.2 Method 4: Mastering the Box Model

METHOD **Computing the space occupied by an element**

Every HTML element is a rectangular box made of four layers, from inside to outside: **content**, **padding** (inner space), **border**, **margin** (outer space separating it from neighbours).

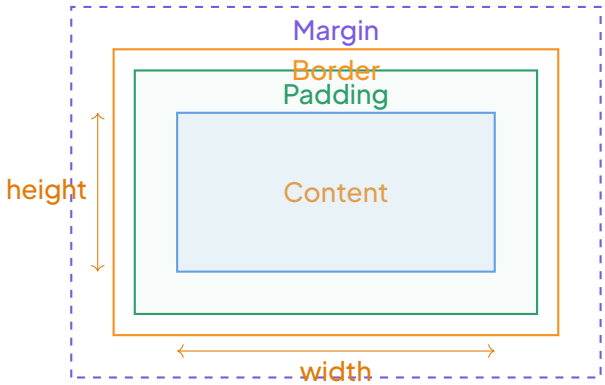
1. Identify the four values: content width, padding, border thickness, margin.
2. Compute the total width occupied (default `box-sizing: content-box`):

$$\text{total width} = \text{content} + 2 \times \text{padding} + 2 \times \text{border} + 2 \times \text{margin}$$

(the left and right values are added, hence the factor 2). The same formula applies vertically for the total height.

3. Use `padding: 10px;` for space inside the box (the background colour extends over it) and `margin: 10px;` for space outside (always transparent).
4. To centre a block horizontally, give it a width and set `margin: auto;`.
5. Modern practice: set `box-sizing: border-box;` on all elements (via `* { box-sizing: border-box; }`) so that `width` and `height` include padding and border, making layout calculations intuitive.

Reflex: padding touches the content; margin touches the neighbours.



The CSS box model: content, padding, border, margin.

WORKED EXAMPLE 7 **Width of a price banner**

A shop in Bamenda displays a banner styled by:

```
.banner { width: 200px; padding: 10px; border: 5px solid black; margin: 20px;
                                                                        }
```

1. Compute the total horizontal space occupied by the banner.
2. The webmaster wants the banner to occupy exactly 200 pixels in total. What content width must be set, keeping the other values?

SOLUTION

1. Total width. Apply the box model formula. On each side we have padding 10 px, border 5 px and margin 20 px, i.e. 10 + 5 + 20 = 35 px per side:

$$\text{total} = 200 + 2 \times 10 + 2 \times 5 + 2 \times 20 = 200 + 20 + 10 + 40 = 270 \text{ px}.$$

The banner therefore occupies 270 pixels horizontally on the screen, although its declared `width` is only 200 px.
2. Adjusting the content width. We now want:

$$w + 2 \times 10 + 2 \times 5 + 2 \times 20 = 200$$

$$w + 70 = 200 \quad \Longrightarrow \quad w = 130 \text{ px}.$$

The webmaster must write `width: 130px;`. Note that the visible box (content + padding + border, without the transparent margin) is then 130 + 20 + 10 = 160 px; the margin completes the 200 px of occupied space.

9 JAVASCRIPT FUNDAMENTALS: SYNTAX, VARIABLES AND CONTROL STRUCTURES

JavaScript is the programming language of the web. It runs in the browser, manipulates the DOM, handles events, and communicates with servers. We use `<script>` tags (usually just before `</body>`) or an external `.js` file loaded with `<script src="app.js"></script>`.

DID YOU KNOW? **JavaScript in Cameroon**

Mobile money USSD simulators, MTN MoMo and Orange Money web apps, and the GCE Board's online registration portal all rely heavily on JavaScript for client-side validation and dynamic updates.

9.1 Method 5: Declaring Variables and Using Control Structures

METHOD **Variables, conditions and loops**

1. Declare variables with `let` (reassignable) or `const` (constant reference). Avoid the old `var` (function-scoped, hoisted).
2. Distinguish the primitive types: number (`2500`, `3.14`, `NaN`, `Infinity`), string (`"Yaoundé"`, `'Douala'`, `Bafoussam̀`), boolean (`true`/`false`), null, undefined. Objects (arrays, functions, dates) are reference types.
3. Use `===` (strict equality) and `!==` (strict inequality) to avoid type coercion surprises. `==` and `!=` perform type conversion and are a frequent source of bugs.
4. For a decision, use `if (condition) { ... } else { ... }` with comparison opera-

tors <, <=, >, >=, ===, !== and logical operators && (and), || (or), ! (not).

5. For repetition, use `for (let i = 0; i < n; i++) { ... }` or `while (condition) { ... }`. In a `while` loop, ensure the condition eventually becomes false, otherwise the loop is infinite.
6. Display a quick result with `alert(...)` or write to the console with `console.log(...)`.

Reflex: = assigns a value, === compares values and types. Confusing them inside an `if` is the most common beginner's error.

WARNING **Type coercion traps**

- `"5" + 2` → `"52"` (string concatenation).
- `"5" - 2` → `3` (numeric subtraction).
- `[] == ""` → `true`, but `[] === ""` → `false`.
- Always convert form inputs with `Number(...)` or `parseInt(...)` before arithmetic.

WORKED EXAMPLE 8 **Mobile money withdrawal fee**

A script computes the fee charged on a mobile money withdrawal: the fee is 1 % of the amount, but it is at least 50 FCFA. Write the JavaScript code that declares an `amount` of 4000 FCFA, computes the fee, and displays it with `alert`.

SOLUTION

Analysis. 1 % of 4000 is 4000 × 0.01 = 40 FCFA, which is below the minimum of 50 FCFA, so the fee must be 50 FCFA. We need a conditional structure.
Code.

```
let amount = 4000;
let fee = amount * 0.01;
if (fee < 50) { fee = 50; }
alert("Fee: " + fee + " FCFA");
```

Line-by-line explanation.

- `let amount = 4000;` declares a numeric variable holding the withdrawal amount.
- `let fee = amount * 0.01;` computes 1 % of the amount; here `fee` receives 40.
- The condition `fee < 50` is true (40 < 50), so the block is executed and `fee` is overwritten with the minimum value 50.

- `alert(...)` opens a dialog box. The + operator joins (concatenates) strings and numbers, producing the message "Fee: 50 FCFA".

Check: for an amount of 20000 FCFA, the fee would be 20000 × 0.01 = 200 FCFA, the condition would be false, and the displayed fee would remain 200 FCFA — the code behaves correctly in both cases.

9.2 Method 6: Combining Loops and the DOM to Generate Content

METHOD **Building repeated content with a loop**

When a page must display a list (timetable, price list, results), do not write the HTML by hand: generate it.

1. Prepare a container in the HTML, e.g. `<ul id="list"></ul>` or `<tbody id="rows"></tbody>`.
2. In the script, select the container.
3. Loop over the data with `for` or `forEach`; at each iteration, build the new content, e.g. `container.innerHTML += "<li>" + item + "</li>";` or create elements with `document.createElement` and `appendChild`.
4. Alternatively, perform a repeated calculation (multiplication table, cumulative total) and display the final result in one element.

Reflex: initialise an accumulator (total = `0`, string = `""`) before the loop, never inside it.

WORKED EXAMPLE 9 **Total of a market basket**

A page helps a trader at Mokolo market. It contains `<p id="total"></p>`. The prices (in FCFA) of the items bought are stored in an array: `let prices = [500, 1500, 250, 1000];`. Write the script that computes the total with a loop and displays "Total: 3250 FCFA" in the paragraph.

SOLUTION

Code.

```
let prices = [500, 1500, 250, 1000];
let total = 0;
for (let i = 0; i < prices.length; i++) {
\quad total = total + prices[i];
}
document.getElementById("total").textContent = "Total: " + total + " FCFA";
```

Step-by-step trace.

- `total` is initialised to `0` before the loop — the accumulator reflex.
- The loop runs with `i` from `0` to `3` (since `prices.length` is 4). The condition `i < prices.length` makes the code work for any number of items, not just four.
- Iteration 1: `total = 0 + 500 = 500`. Iteration 2: `total = 500 + 1500 = 2000`. Iteration 3: `total = 2000 + 250 = 2250`. Iteration 4: `total = 2250 + 1000 = 3250`.
- After the loop, the paragraph's text is set by concatenation: "Total: 3250 FCFA".

Why not hard-code the four values? Because with `prices.length` the same script works tomorrow when the trader buys ten items. This pair "array + loop + accumulator + DOM output" summarises the whole chapter: CSS for presentation, HTML for structure, and JavaScript for behaviour — the three pillars of every interactive web page you will meet at the GCE and beyond.

10 INTERACTIVE PAGES: DOM MANIPULATION, EVENTS AND FORM VALIDATION

10.1 Method 7: Manipulating the DOM

METHOD **Reading and modifying page content with JavaScript**

The **DOM** (Document Object Model) represents the page as a tree of objects that JavaScript can read and modify.

1. Select the element: `document.getElementById("titre")` for an id, `document.querySelector(".info")` for the first element matching a CSS selector, `document.querySelectorAll(".info")` for a `NodeList` of all matches.
2. Read or change its text: `element.textContent = "New text";` (safe, ignores HTML) or `element.innerHTML = "<strong>New</strong> text";` (parses HTML, use with caution).
3. Change its style: `element.style.color = "red";` (CSS properties with a hyphen become camelCase: `font-size` → `fontSize`, `background-color` → `backgroundColor`).
4. Modify classes: `element.classList.add("active")`, `remove("hidden")`, `toggle("open")`, `contains("active")`.
5. Read a form field: `document.getElementById("nom").value` — the result is always a string; convert it with `Number(...)` or `parseInt(...)` before any calculation.

6. Make sure the script runs after the HTML elements exist (place the `<script>` tag at the end of the body, or use `defer` attribute), otherwise the selection returns `null`.

Reflex: select first, then act. Always test the selection with `console.log(element)` when something fails.

WORKED EXAMPLE 10 **A button that updates a school notice board**

The page of a college in Buea contains:

```
<h2 id="notice">Classes start at 7:30.</h2>
<button id="btn">Update</button>
```

Write the JavaScript code so that clicking the button changes the heading text to "Classes start at 8:00." and colours it green.

SOLUTION

Code.

```
let notice = document.getElementById("notice");
let btn = document.getElementById("btn");
btn.addEventListener("click", function() {
\quad notice.textContent = "Classes start at 8:00.";
\quad notice.style.color = "green";
});
```

Explanation.

- The first two lines select the two elements by their ids and store them in variables, so we do not search the document repeatedly.
- `addEventListener("click", function() { ... })` attaches an **event handler**: the anonymous function is executed each time the button is clicked. This is the heart of interactivity: the page reacts to the user.
- Inside the handler, `textContent` replaces the text of the heading, and `style.color` applies the green colour immediately, without reloading the page.

Common trap: writing `notice.style.font-colour` or `notice.color` would fail; DOM style properties use camelCase names (`backgroundColor`, `fontSize`) and must be reached through `style`.

10.2 Method 8: Handling Events and Validating a Form

METHOD **Validating user input before submission**

Form validation prevents empty or absurd data from being sent — think of an e-registration form on a school website.

1. Give each input an `id` and select it with `getElementById`.
2. Listen to the form's `submit` event (or a button's `click` event).
3. Read the values with `.value`; check them: empty string, length, numeric range, `isNaN(...)` for "is not a number".
4. If a check fails: display an error message (in a dedicated `<span>` or with `alert`) and stop the submission with `event.preventDefault();`.
5. If all checks pass, let the form be submitted or display a confirmation.
6. HTML5 built-in validation: use `required`, `min`, `max`, `pattern`, `type="email|number|tel"` for a first line of defence without JavaScript.

Reflex: never trust the user. Validate every field, and remember that `.value` returns a string: `"12" + "5"` gives `"125"`, not 17.

WORKED EXAMPLE 11 **Validating a GCE registration form**

A school website contains a form with two fields: `<input id="cand" type="text">` for the candidate's name and `<input id="age" type="text">` for the age, plus a submit button. Write the JavaScript code that, on submission:

1. refuses an empty name;
2. refuses an age that is not a number or is below 14;
3. displays "Registration accepted" otherwise.

SOLUTION

Code.

```
let form = document.querySelector("form");
form.addEventListener("submit", function(event) {
\quad let name = document.getElementById("cand").value;
\quad let age = Number(document.getElementById("age").value);
\quad if (name == "") {
\quad\quad alert("Please enter the candidate's name.");
\quad\quad event.preventDefault();
\quad } else if (isNaN(age) || age < 14) {
\quad\quad alert("Please enter a valid age (14 or more).");
```

```
\quad\quad event.preventDefault();
\quad } else {
\quad\quad alert("Registration accepted");
\quad }
});
```

Explanation.

- We listen to the `submit` event of the whole form, so validation runs whether the user clicks the button or presses Enter.
- `Number(...)` converts the string read from the age field into a number; if the user typed letters, the result is `NaN`, detected by `isNaN(age)`.
- The first `if` tests the empty string `""`. The `else if` combines two rejections with the logical OR `||`: non-numeric age or age under 14.
- `event.preventDefault()` cancels the browser's default action (sending the form), so invalid data never leaves the page.
- Only when both tests pass does the final `else` display the confirmation message.

Test cases: name "Amina", age "17" → accepted; name "" → first alert; name "Amina", age "abc" → `NaN`, second alert; age "12" → second alert. All branches are covered, which is exactly what a GCE examiner expects to see.

EXPERIMENT **Build a dynamic multiplication table**

Create a page with an input `<input id="n" type="number" min="1" max="12">`, a button `<button id="go">Generate</button>`, and an empty `<table id="tbl"></table>`. Write a script that, on click, reads `n`, clears the table, and uses a loop to append rows `<tr><td>1 x n = result</td></tr>` up to `12 x n`. Use `document.createElement` and `appendChild` for clean DOM manipulation.

11 EXERCISES

11.1 I check my knowledge

EXERCISE 5 **Multiple choice questions**

For each question, choose the single correct answer.

1. Which of the following is the correct CSS syntax to make all `<h1>` headings red?

   (a) `h1 {color: red;}`

   (b) `{h1: color=red;}`

   (c) `h1: red color;`

   (d) `color h1 {red;}`

2. Which CSS selector has the highest specificity?

   (a) An element selector such as `p`

   (b) A class selector such as `.note`

   (c) An id selector such as `#title`

   (d) A pseudo-class such as `:hover`

3. In JavaScript, which keyword declares a variable whose value cannot be reassigned?

   (a) `var`

   (b) `let`

   (c) `const`

   (d) `static`

4. Which JavaScript instruction changes the text inside the element `<p id="msg">`?

   (a) `document.getElementById("msg").innerHTML = "Hello";`

   (b) `getElementById.msg = "Hello";`

   (c) `document.msg.innerHTML("Hello");`

   (d) `msg.document.text = "Hello";`

EXERCISE 6 **True or false — justify**

State whether each statement is true or false, and justify your answer in one or two sentences.

1. An inline style written in a `style` attribute overrides a rule in an external style sheet.
2. In the CSS box model, the margin is the space inside the border of an element.
3. In JavaScript, the operator = compares two values for equality.
4. JavaScript code placed in a web page is executed by the browser, not by the web server.

EXERCISE 7 **Definitions**

Define each of the following terms in your own words, giving one example for each:

1. **CSS selector**;
2. **CSS declaration**;
3. **DOM (Document Object Model)**;
4. **JavaScript event**.

EXERCISE 8 **Direct calculations**

1. A CSS rule sets `width: 200px; padding: 10px; border: 2px solid black; margin: 15px;` on a box. Compute the total horizontal space occupied by the box, from the outer edge of the left margin to the outer edge of the right margin.
2. Compute the specificity score (in the form a-b-c: ids, classes, elements) of each of the following selectors: `p`, `.menu li`, `#header h1`. Which one wins if they all target the same heading?

11.2 I practise

EXERCISE 9 **The cascade in action**

A web page contains the paragraph `<p id="intro" style="color: green;">Welcome</p>`. The external style sheet contains `p {color: blue;}` and the internal `<style>` block in the `<head>` contains `#intro {color: red;}`.

1. State the final colour in which the paragraph is displayed.

2. Justify your answer by explaining the roles of the cascade (origin order) and of specificity.
3. What would the colour be if the inline `style` attribute were removed?

EXERCISE 10 **Styling a results table**

The computer science teacher of GBHS Bamenda displays term results in an HTML table of class `results`. Write the CSS rules to:

1. give the header row (`th`) a blue background with white bold text;
2. give every cell (`th` and `td`) a 1px solid black border;
3. centre the content of every cell horizontally;
4. collapse the borders so that adjacent cells share a single border line.

EXERCISE 11 **Tracing a script**

Consider the following JavaScript script:

```
let total = 0;
for (let i = 1; i <= 5; i++) {
\quad if (i % 2 == 0) { total = total + i; }
}
alert(total);
```

1. Trace the execution of the loop step by step (value of `i`, test result, value of `total`).
2. Write exactly what the `alert` displays.
3. The programmer had written `if (i = 2)` instead of `if (i % 2 == 0)`. Explain the error caused by using = instead of ==, and state what happens.

EXERCISE 12 **Change Theme button**

A page contains `<button id="themeBtn">Change Theme</button>`. Write a complete JavaScript script (using `getElementById`, an event listener, and `style.backgroundColor`) such that each click on the button switches the page background between white and dark grey (`#333333`). Hint: keep track of the current state with a Boolean variable.

11.3 I prepare for the GCE

EXERCISE 13 **Structured question: CSS and JavaScript**

A school in Yaoundé wants a modern website presenting its classes, results and announcements.

1. Define CSS and JavaScript, stating the role of each in a web page. (4 marks)
2. Give two differences between CSS and JavaScript (nature of the language, purpose, way of acting on the page). (4 marks)
3. Describe one concrete situation in the school website where CSS is the appropriate tool, and one where JavaScript is required. Justify each choice. (4 marks)

EXERCISE 14 **Form validation problem**

The school's registration page contains the form:

```
<form id="regForm">
\quad <input type="text" id="phone" placeholder="Phone number">
\quad <input type="submit" value="Register">
</form>
```

Parents must enter a Cameroonian phone number of exactly 9 digits (for example `650123456`).

1. Explain why validation on the client side improves the user experience, and why it does not replace server-side validation. (4 marks)
2. Complete the function `checkForm()` below so that it reads the phone field, verifies that it contains exactly 9 digits (characters `0–9` only), displays an alert if invalid, and returns `true` if valid or `false` otherwise. (6 marks)

```
function checkForm() {
\quad let phone = document.getElementById("phone").value;
\quad // to be completed
}
```

3. Show how the function is attached to the form's `submit` event, and explain the role of `event.preventDefault()`. (4 marks)

EXERCISE 15 **Design study: an interactive notice board**

The principal wants the school's online notice board to: (i) have a coloured banner, styled headings and alternating row colours in the announcements table; (ii) allow a visitor to click a button to hide or show the details of each announcement; (iii) refuse empty announcement titles when the secretary adds a notice.

1. For each requirement (i), (ii) and (iii), state whether it is achieved with HTML, CSS or JavaScript, and justify. (6 marks)
2. Write the CSS rule producing alternating background colours on the rows of a table of class `board` (use the `:nth-child(even)` pseudo-class). (3 marks)
3. Outline, in pseudocode or commented JavaScript, the mechanism used for requirement (ii): which DOM method selects the element, which event is listened to, and which style property is toggled. (5 marks)
4. Give two advantages of placing the CSS in an external file rather than inline for a site maintained by several teachers. (2 marks)

12 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — Multiple choice questions**

1. Answer: a) `h1 {color: red;}`. A CSS rule consists of a **selector** (`h1`) followed by a **declaration block** in curly braces; each declaration is a `property: value;` pair. Options b), c) and d) mix up the order or use = instead of `:`.
2. Answer: c) An id selector such as `#title`. Specificity increases in the order: element (0-0-1) < class/pseudo-class (0-1-0) < id (1-0-0). The id selector therefore wins over the others.
3. Answer: c) `const`. A variable declared with `const` cannot be reassigned after initialisation; `var` and `let` allow reassignment, and `static` is not a JavaScript variable declaration keyword.
4. Answer: a) `document.getElementById("msg").innerHTML = "Hello";`. The DOM method `getElementById` belongs to the `document` object and returns the element whose `id` is `"msg"`; setting its `innerHTML` property replaces its content.

SOLUTION **Exercise 2 — True or false — justify**

1. True. In the CSS cascade, an inline style (in a `style` attribute) has higher priority than rules from internal or external style sheets, whatever their specificity (unless `!important` is used).
2. False. The margin is the space outside the border, separating the element from its neighbours. The space inside the border, between the content and the border, is the **padding**.
3. False. The single = is the assignment operator: it stores a value in a variable. Comparison for equality uses == (loose equality) or === (strict equality, value and type).
4. True. JavaScript embedded in a web page is a **client-side** language: it is downloaded with the page and executed by the browser's JavaScript engine on the visitor's computer, not by the web server.

SOLUTION **Exercise 3 — Definitions**

1. CSS selector: the part of a CSS rule that indicates which HTML elements the rule ap-

plies to. Example: in `p.note {color: blue;}`, the selector `p.note` targets all paragraphs of class `note`.

2. CSS declaration: a `property: value;` pair inside a declaration block; it states one presentation instruction. Example: `font-size: 14px;` is a declaration setting the text size to 14 pixels.
3. DOM (Document Object Model): the tree-shaped representation of a web page that the browser builds in memory, in which every element, attribute and text is a node (object) that JavaScript can read and modify. Example: `document.getElementById("msg")` accesses the node of the element with id `msg` in the DOM tree.
4. JavaScript event: an action or occurrence detected by the browser (a click, a key press, the loading of the page, a form submission) to which a script can react by executing a function called an event handler. Example: the `click` event on a button can trigger a function that changes the page background.

SOLUTION **Exercise 4 — Direct calculations**

1. In the standard box model, the total horizontal space is the sum of margin, border, padding and width on both sides:

$$\text{Total} = \text{margin} + \text{border} + \text{padding} + \text{width} + \text{padding} + \text{border} + \text{margin}$$
$$= 15 + 2 + 10 + 200 + 10 + 2 + 15 = 254 \text{ px}.$$

   The box occupies a total horizontal space of 254 px.

2. Specificity scores (ids-classes-elements):

Selector	Score	Counting
`p`	0-0-1	1 element
`.menu li`	0-1-1	1 class + 1 element
`#header h1`	1-0-1	1 id + 1 element

   Comparing from the left: 1 > 0 in the id column, so `#header h1` has the highest specificity and wins if all three target the same heading.

SOLUTION **Exercise 5 — The cascade in action**

1. The paragraph is displayed in green.
2. When several rules apply to the same element, the browser first compares origin and priority: an **inline style** (the `style` attribute) beats rules from internal and external style sheets, regardless of their specificity. Here the inline declaration `color: green` therefore overrides both `#intro {color: red;}` (internal) and `p {color: blue;}` (external). Specificity is only used to decide between rules of the same origin: between the two style-sheet rules, the id selector `#intro` (1-0-0) is more specific than the element selector `p` (0-0-1), so red would beat blue.
3. Without the inline `style` attribute, the decision falls back to specificity between the two remaining rules: `#intro` (1-0-0) wins over `p` (0-0-1), so the paragraph would be displayed in red.

SOLUTION **Exercise 6 — Styling a results table**

The required CSS rules are:

```
table.results { border-collapse: collapse; }
.results th { background-color: blue; color: white; font-weight: bold; }
.results th, .results td { border: 1px solid black; text-align: center; }
```

Explanation:

1. `.results th` targets the header cells of the table: `background-color: blue` gives the blue background, `color: white` the white text and `font-weight: bold` the bold style.
2. The grouped selector `.results th, .results td` applies the same `1px solid black` border to every cell, header or data.
3. `text-align: center` in the same rule centres the content of every cell horizontally.
4. `border-collapse: collapse` on the table makes adjacent cells share a single border line instead of two separate borders.

SOLUTION **Exercise 7 — Tracing a script**

1. Trace of the loop (the condition `i % 2 == 0` is true when `i` is even):

i	i <= 5	i % 2 == 0	total after the iteration
1	true	false	0
2	true	true	0 + 2 = 2
3	true	false	2
4	true	true	2 + 4 = 6
5	true	false	6
6	false	loop ends	

2. The `alert` displays exactly 6 (the sum of the even numbers from 1 to 5: 2 + 4).
3. Writing `if (i = 2)` is an assignment, not a comparison: the value `2` is stored into `i`, and the condition evaluates to `2`, which is truthy (any non-zero value is treated as true). Two consequences follow: the branch is always executed, and `i` is reset to 2 at every test, so after `i++` it becomes 3, then is set back to 2 again: the loop never ends (infinite loop) and the browser tab freezes. The correct comparison operators are `==` or `===`.

SOLUTION **Exercise 8 — Change Theme button**

Complete script:

```
let dark = false; // current state: false = white, true = dark grey
let btn = document.getElementById("themeBtn");
btn.addEventListener("click", function() {
\quad dark = !dark; // switch the state
\quad if (dark) {
\quad\quad document.body.style.backgroundColor = "#333333";
\quad } else {
\quad\quad document.body.style.backgroundColor = "white";
\quad }
});
```

Explanation: `getElementById("themeBtn")` selects the button in the DOM; `addEventListener("click", ...)` attaches a handler executed at each click; the Boolean `dark` remembers the current theme and is inverted with `!dark`; finally `document.body.style.backgroundColor` applies the corresponding colour to the page background.

SOLUTION **Exercise 9 — Structured question: CSS and JavaScript**

(a) Definitions and roles (4 marks).

- CSS (Cascading Style Sheets): a style-sheet language used to describe the presentation of an HTML page — colours, fonts, sizes, spacing, layout. (1 mark definition + 1 mark role)
- JavaScript: a programming (scripting) language executed by the browser, used to add behaviour and interactivity to a page — reacting to events, modifying content dynamically, validating forms. (1 mark definition + 1 mark role)

(b) Two differences (4 marks, 2 marks each).

Criterion	CSS	JavaScript
Nature	Declarative style language (selector + declarations)	Full programming language (variables, tests, loops, functions)
Purpose	Appearance and layout of the page	Logic, calculations and interactivity
Way of acting	Applies static presentation rules to elements	Modifies the DOM dynamically in response to events

Any two of these rows earn the 4 marks. (c) Concrete situations (4 marks, 2 marks each).

- CSS is appropriate for presenting the results table with the school's colours, styled headings and centred columns: this is a purely visual formatting task, which is exactly the role of CSS.
- JavaScript is required to check that a phone number entered in the registration form contains 9 digits before sending it: this requires a test (logic) and a reaction to the submit event, which CSS cannot do.

Examiner's expectations: precise definitions naming the type of language; differences expressed as clear contrasts, not vague statements; situations must be justified by the nature of the task (presentation vs. logic/interaction).

SOLUTION **Exercise 10 — Form validation problem**

(a) Client-side validation (4 marks). It improves the user experience because the check is immediate (no round trip to the server), the user is warned at once and can correct the input, and useless network traffic and server load are reduced. (2 marks)
However, it does not replace server-side validation: JavaScript can be disabled or bypassed in the browser, so a malicious or erroneous request can still reach the server; the server must therefore re-check every piece of data before storing it. (2 marks)
(b) Completed function (6 marks).

```
function checkForm() {
\quad let phone = document.getElementById("phone").value;
\quad if (phone.length != 9) {
\quad\quad alert("The number must contain exactly 9 digits.");
\quad\quad return false;
\quad }
\quad for (let i = 0; i < phone.length; i++) {
\quad\quad if (phone[i] < "0" || phone[i] > "9") {
\quad\quad\quad alert("Only digits 0-9 are allowed.");
\quad\quad\quad return false;
\quad\quad }
\quad }
\quad return true;
}
```

Mark scheme: reading the field with `.value` (given); testing the length equal to 9 (2 marks); checking each character is between `"0"` and `"9"` with a loop (2 marks); displaying an `alert` and returning `false` when invalid (1 mark); returning `true` when valid (1 mark). (A solution using a regular expression such as `/^[0-9]{9}$/.test(phone)` earns full credit.)
(c) Attachment to the submit event (4 marks).

```
document.getElementById("regForm").addEventListener("submit", function(event) {
\quad if (!checkForm()) {
\quad\quad event.preventDefault();
\quad }
});
```

The listener is attached to the form's `submit` event (2 marks). `event.preventDefault()` cancels the browser's default action — sending the form to the server — so that an invalid form is not submitted (2 marks). When `checkForm()` returns `true`, the default action proceeds normally.

SOLUTION **Exercise 11 — Design study: an interactive notice board**

(a) Technology for each requirement (6 marks, 2 each).

- (i) CSS. Coloured banner, styled headings and alternating row colours are pure presentation; CSS applies them through selectors and declarations without any pro-

gramming logic.

- (ii) JavaScript. Hiding or showing details when the visitor clicks requires reacting to the `click` event and modifying the DOM dynamically: this is behaviour, the role of JavaScript.
- (iii) JavaScript. Refusing an empty title means testing the content of a field when the form is submitted and blocking the submission: this decision logic cannot be expressed in HTML or CSS alone (HTML's `required` attribute may be mentioned as a complement, but the programmed check is JavaScript's role).

(b) Alternating rows (3 marks).

```
.board tr:nth-child(even) { background-color: lightgray; }
```

The pseudo-class `:nth-child(even)` selects every even-numbered row of the table of class `board` and paints its background (1 mark for the selector `.board tr`, 1 mark for `:nth-child(even)`, 1 mark for the declaration). (c) Mechanism for hide/show (5 marks).

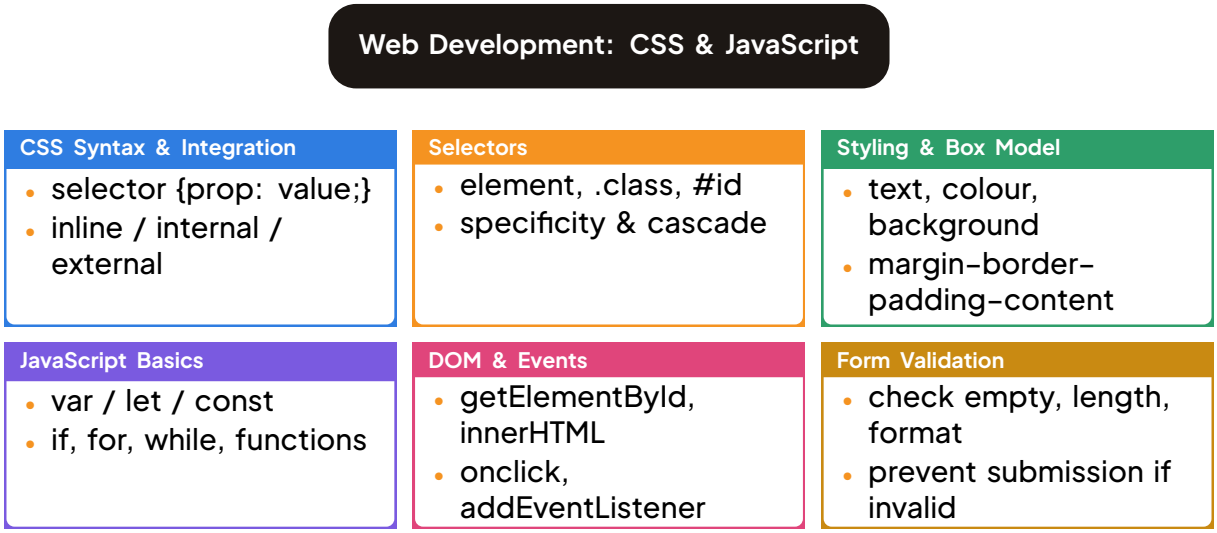
```
// select the button and the details block in the DOM
let btn = document.getElementById("detailsBtn");
let details = document.getElementById("details");
// listen to the click event on the button
btn.addEventListener("click", function() {
\quad // toggle the display property
\quad if (details.style.display == "none") {
\quad\quad details.style.display = "block";
\quad } else {
\quad\quad details.style.display = "none";
\quad }
});
```

Expected points: the DOM selection method `getElementById` (1 mark); the event listened to is `click` via `addEventListener` (2 marks); the toggled style property is `display` (between `none` and `block`) (2 marks). (d) Advantages of an external style sheet (2 marks, 1 each).

- Consistency and single maintenance point: one file styles the whole site; a teacher edits it once and every page is updated, avoiding contradictory inline styles.
- Separation of concerns and reusability: HTML content and presentation are kept apart, pages are lighter and easier to read, and the same style sheet is shared (and cached) by all pages.

Examiner's expectations: each justification must link the requirement to the nature of the technology (presentation vs. event-driven logic); code excerpts must use correct syntax; short, precise answers score full marks.

13 SUMMARY SHEET



Mind map of the chapter: styling pages with CSS and adding interactivity with JavaScript.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- **CSS** (Cascading Style Sheets): language that describes the presentation (colours, fonts, layout) of an HTML document, separating content from appearance.
- **Declaration**: a pair `property: value;` (e.g. `color: blue;`). A **rule** = selector + declaration block.
- **Selector**: pattern that targets elements: element (`p`), class (`.menu`), id (`#top`), descendant (`div p`), pseudo-class (`a:hover`).
- **Specificity**: priority order used when rules conflict: inline style > id > class > element; the last rule wins at equal specificity (the cascade).
- **Box model**: every element is a box: content → padding → border → margin. Total width = content + padding + border + margin.
- **JavaScript**: scripting language executed by the browser that makes pages dynamic and interactive.
- **DOM** (Document Object Model): tree representation of the page that JavaScript can read and modify.
- **Event**: action detected by the browser (click, key press, submit) that can trigger a function.

Essential syntax to know

- CSS rule: `h1 { color: #0066CC; text-align: center; }`
- Integration: inline (`style="..."`), internal (`<style>` in `<head>`), external (`<link rel="stylesheet" href="style.css">`) — the external sheet is the recommended method.
- Colours: name, hexadecimal (`#FF0000`), `rgb(255,0,0)`.
- JavaScript: `let x = 5; const PI = 3.14;` — strings in quotes, comments with `//`.
- Conditions and loops: `if (age >= 18) {...} else {...};` `for (let i=0; i<10; i++) {...};` `while (n > 0) {...}`.
- Function: `function greet(name) { return "Hello " + name; }`
- DOM access: `document.getElementById("msg").innerHTML = "Welcome";`
- Event handling: `<button onclick="greet('Ali')">` or `element.addEventListener("click", greet);`
- Validation: read `document.getElementById("email").value`, test it, show an error message and `return false;` to block submission.

Methods

- To style a page: identify elements → choose selectors → write rules in an external sheet → link it in `<head>` → test in the browser.
- To add interactivity: write the function → attach it to an event → test with the browser console.
- To validate a form: check each field (empty? length? format?) → display a clear message → allow submission only if all tests pass.

Common mistakes

- Forgetting the semicolon between CSS declarations or the braces around the block.
- Confusing `.class` (reusable, many elements) with `#id` (unique element).
- Using = (assignment) instead of == or === (comparison) in an `if`.
- Case sensitivity: `getElementById` is not `getElementByID`; JavaScript and CSS ids must match exactly.
- Placing the script before the HTML elements it manipulates (put it at the end of `<body>` or use `onload`).
- Confusing padding (inside the border) with margin (outside the border).

GCE tips

- Be able to read and complete a short CSS or JavaScript extract, and to predict the output of a script (typical structured question).

- Know the three CSS integration methods and be ready to justify why external sheets are best (reuse, maintenance, separation of concerns).
- Draw and label the box model quickly; a labelled diagram earns full marks.
- For validation questions, always mention client-side checking with JavaScript and that it improves user experience (e.g. a student registration form on a school website).
- Write code neatly, indent blocks, and give meaningful identifiers — clarity is rewarded.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+